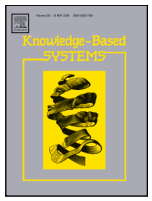
Knowledge-Based Systems 330 (2025) 114471



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





Enhancing leadership-based metaheuristics using reinforcement learning: A case study in grey wolf optimizer

Afifeh Maleki [a], Mehdy Roayaei [a,*], Seyedali Mirjalili [b]

[a] Department of Electrical and Computer Engineering, Tarbiat Modares University, Tehran, Iran
[b] Centre for Artificial Intelligence Research and Optimization, Torrens University Australia, Fortitude Valley, 4006, QLD, Australia

ARTICLE INFO

Keywords:
Grey wolf optimizer
Reinforcement learning
Exploration-exploitation balance
Leader update mechanism

ABSTRACT

Metaheuristics are widely applied in optimization because of their flexibility and ability to address complex and high-dimensional problems. Nevertheless, they face persistent challenges, including susceptibility to local optima, limited parameter adaptability, and premature convergence. Leadership-based metaheuristics, in which leaders guide the search process, encounter additional difficulties such as limited exploration capacity, leader stagnation, and reduced diversity, often stemming from underutilization of data generated during the search. To overcome these limitations, this study proposes a reinforcement learning–based approach, RL-LGWO, which enhances the Grey Wolf Optimizer (GWO) by integrating multi-agent reinforcement learning. In RL-LGWO, agents share experiences to improve decision-making, and reinforcement learning is employed to decouple and adapt the leader update mechanism, thereby improving the exploration–exploitation balance and enabling leaders to dynamically escape local optima. The proposed method was evaluated against two GWO-enhancing algorithms, three RL-based GWO variants, PSO, WOA, and the original GWO across 23 well-known benchmark functions, in addition to the recent CEC2022 benchmark suite. Experimental results show that RL-LGWO achieved the best solutions on 17 of the 23 benchmark functions, with superior convergence speed and improved stability, while incurring only a minor runtime increase compared with the original GWO. Furthermore, on the CEC2022 suite, RL-LGWO outperformed competing algorithms on 10 of 12 test functions, underscoring its robustness and adaptability to recent and challenging benchmarks. Overall, the findings indicate that RL-LGWO delivers a substantive improvement over state-of-the-art alternatives and holds strong potential to advance leadership-based metaheuristics for a wide range of optimization problems.

1. Introduction

Metaheuristics are powerful optimization techniques widely applied to solve complex and multi-dimensional problems in various domains, including engineering, economics, and artificial intelligence [1]. Unlike traditional optimization methods, metaheuristics do not require the problem to be convex or differentiable, making them highly versatile in tackling real-world problems with uncertain, noisy, or complex search spaces. Among these algorithms, the Grey Wolf Optimizer (GWO) has gained popularity due to its simple structure, computational efficiency, and competitive performance [2]. This enables GWO to perform well in various scientific and industrial applications [3,4].

Leadership-based metaheuristics represent an important class of optimization algorithms that utilize a leader or group of leaders to guide the search process. These algorithms organize the population into hierarchical structures, where leaders play a pivotal role in balancing exploration and exploitation [5]. Notable examples of leadership-based algorithms include GWO, Particle Swarm Optimization (PSO), and Whale Optimization Algorithm (WOA). Inspired by natural and social phenomena, such as the hunting strategies of wolves and the bubble-net feeding behavior of humpback whales, these algorithms effectively mimic real-world collective behaviors. This capability makes them highly suitable for addressing a diverse range of optimization challenges, particularly in complex, multi-dimensional, and noisy environments [6].

Leadership-based metaheuristics, despite their strengths, face significant challenges that can hinder their effectiveness. One key issue is maintaining an optimal balance between exploration and exploitation. Striking this balance is particularly challenging in complex, high-dimensional, and multi-modal search spaces, where insufficient exploration can lead to suboptimal solutions, and excessive exploitation risks premature convergence [7,8]. Another critical challenge arises from the central role of leaders in guiding the population. If the leaders are not properly initialized or fail to identify and explore promising regions, the entire population is misdirected, as their movements depend on the

* Corresponding author.
E-mail addresses: m.afifeh@modares.ac.ir (A. Maleki), mroayaei@modares.ac.ir (M. Roayaei), ali.mirjalil@Torrens.edu.au (S. Mirjalili).

https://doi.org/10.1016/j.knosys.2025.114471
Received 14 March 2025; Received in revised form 27 August 2025; Accepted 10 September 2025
Available online 16 September 2025
0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

guidance of the leaders. This misalignment not only disrupts the balance between exploration and exploitation but also increases the likelihood of stagnation in local optima [9], ultimately limiting the algorithm's ability to find global solutions [10].

While traditional metaheuristics typically do not leverage the data generated during the optimization process to improve their performance or adapt to changing conditions, recent advancements in data-driven metaheuristics have introduced mechanisms that utilize search data for self-adaptation. For example, some algorithms analyze the collected data to dynamically adjust parameters or modify their search strategies to escape local optima, enhancing robustness and performance [11].

Reinforcement Learning (RL) is a machine learning approach that enables agents to learn optimal behaviors through interactions with their environment, leveraging rewards or penalties as feedback to improve decision-making [12]. This capability makes RL highly suitable for enhancing leadership-based metaheuristics by utilizing data generated during their optimization process to refine algorithm performance. RL facilitates dynamic parameter adjustment and guides optimal search behaviors, helping the algorithm escape local optima and improve convergence. By addressing the limitations of traditional metaheuristics, RL enhances their adaptability and robustness, improving performance in complex optimization problems.

This paper examines the enhancement of leadership-based metaheuristic algorithms by integrating Reinforcement Learning. The Grey Wolf Optimizer is a widely used metaheuristic algorithm inspired by the social hierarchy and cooperative hunting behavior of grey wolves. Its simple implementation and effective balance between exploration and exploitation have led to successful applications in various fields such as building energy optimization [13], UAV path planning [14], wireless sensor networks [37], image processing [38], feature selection [39], and microwave circuit design [15]. These advantages make GWO a strong candidate for further enhancement. The proposed framework targets improvements in low-level decision-making mechanisms commonly found in such algorithms. To demonstrate its effectiveness and generalizability, the method is applied to the GWO, a well-established leadership-based metaheuristic featuring a hierarchical structure with multiple leaders (alpha, beta, and delta). This multi-leader structure offers a flexible foundation for coordinating follower agents, making GWO a suitable candidate for RL-based enhancement. The overall design allows for easy adaptation to other leader-based algorithms with different leadership configurations.

Q-learning is employed to dynamically select operators during exploration and exploitation, and to adaptively adjust the leader update mechanism. Unlike traditional approaches that rely on fixed or monotonically decreasing exploration–exploitation strategies, the proposed method treats each wolf as an autonomous agent capable of learning from its environment. A shared Q-table aggregates the collective experience of all agents, promoting informed and coordinated decision-making throughout the population. This collaborative learning strategy results in improved convergence speed, robustness, and optimization performance. The contribution of the paper can be stated as follows:

- **Leadership-based Framework:** A novel leadership-based optimization framework is introduced, integrating RL to enhance the performance of leadership-based algorithms. This framework provides a systematic approach for leveraging RL to intelligently guide followers toward leaders, improving adaptability and convergence efficiency. The proposed framework is designed to be generalizable, allowing its application across various leadership-based algorithms.
- **Shared Memory Mechanism:** In contrast to prior studies where each individual maintained an isolated memory table, the proposed approach introduces a unified tabular memory structure that is shared among all individuals. This collective experience-sharing mechanism enhances coordination, supports more informed decision-making, and contributes to improved convergence speed and optimization performance.
- **Decoupling Leader Update Mechanisms:** In contrast to the standard GWO algorithm, which updates all wolves uniformly, the leader update process in the proposed method is formulated as a reinforcement learning problem. This formulation mitigates the risk of leaders becoming trapped in local optima and enhances the overall search and optimization performance by effectively leveraging historical experience and dynamically generated execution data.
- **Adaptive explore-exploit and leader influence coefficient:** A dynamic mechanism is introduced to enable each follower to determine whether to explore or exploit, and according to which leader. The leader influence coefficient dynamically adjusts the follower's decision-making process, ensuring an effective balance between global exploration and local exploitation. This adaptive strategy enhances solution quality and accelerates convergence.

The remainder of the paper is organized as follows. Section 2 reviews related work, categorizing research, identifying strengths and limitations, and highlighting research gaps addressed by the proposed method. Section 3 details the methodology and innovations of the proposed approach. Section 4 evaluates its performance through experiments and comparative analyses using various metrics and benchmarks. Finally, Section 5 summarizes the findings, discusses implications, and suggests directions for future research.

2. Literature review

In recent years, metaheuristics, especially leadership-based ones, have been widely studied for solving complex optimization problems [2]. However, these algorithms face several challenges, including premature convergence and difficulties in parameter tuning, which may significantly limit their performance in complex optimization scenarios. To address these issues, various strategies have been proposed. Among them, the integration of reinforcement learning (RL) into metaheuristic frameworks has been shown to yield promising improvements in both performance and adaptability. Specifically, RL has been employed to enhance exploration–exploitation balance, enable dynamic parameter tuning, guide the selection of search strategies, and improve outcomes in specific application domains. In this section, improvements achieved through the application of reinforcement learning to metaheuristic algorithms are reviewed and categorized accordingly.

Yuvaraj et al. [16] proposed an improved task allocation scheme in serverless computing using a GWO-based reinforcement learning approach, called RIL. Their method effectively reduced execution time and adapted to varying load distributions, leading to improvements in throughput, cost reduction, and latency. However, RIL also presents certain limitations, including increased computational overhead due to reinforcement learning, sensitivity to hyperparameter tuning, and potential scalability challenges in highly dynamic cloud environments.

Lin et al. [17] applied RL to optimize GWO parameters for solving a stochastic flexible job shop scheduling problem, an NP-hard problem. Their approach integrates an optimal computing budget allocation method and a RL-based parameter tuning scheme, enabling reliable and efficient scheduling. However, the proposed method introduces significant computational overhead, making it unsuitable for real-time large-scale scheduling. Additionally, the study is based on two specific manufacturing scenarios, which restricts the generalizability of the approach to diverse scheduling environments. The RL-based tuning scheme requires precise hyperparameter selection and extensive training, limiting its adaptability across different problem instances.

Emary et al. [18] introduced an Enhanced Grey Wolf Optimizer (EGWO), combining RL with Artificial Neural Networks (ANNs). RL was employed to dynamically update the parameters of candidate solutions, while ANNs were used to estimate the fitness of solutions. This hybrid approach enabled better control over exploration and exploitation, yielding superior results when compared to standard algorithms like Genetic Algorithm (GA), PSO, and GWO.

Seyyedabbasi et al. [19] further explored the integration of RL with metaheuristics by combining Q-learning with both GWO and the WOA. Their method improved the balance between the exploration and exploitation phases, leading to better convergence rates. However, one limitation of their approach is that each wolf in the GWO algorithm has its own separate Q-table. As the number of wolves increases, the execution speed of the program tends to decrease because more space is required to store and update the Q-values for each wolf. This results in higher computational overhead, especially when dealing with large-scale problems or when a larger number of wolves is involved, which can significantly impact the overall efficiency and performance of the algorithm.

Beccerra-Rozas [20] utilized the SARSA algorithm to improve WOA by selecting the best binary schemes based on learned experience in each iteration. Similarly, Zhao et al. [21] incorporated Q-learning into the Brain Storm Optimization (BSO) algorithm, using four mutation strategies to enhance search efficiency. Their RLBSO algorithm outperformed other advanced methods on benchmark sets and engineering problems.

Moreover, Sadhu et al. [22] addressed the challenge of parameter tuning in the Firefly Algorithm by proposing an RL-based method for optimizing two key parameters-randomness and light absorption coefficient. Their approach improved the balance between exploration and exploitation, enhancing the algorithm's effectiveness in complex optimization tasks.

Wu et al. [23] proposed a reinforcement learning-enhanced PSO to address the slow convergence issue of classical PSO. They integrated a normal distribution into the velocity update rule, with parameters estimated adaptively via a policy network. While this method significantly accelerated convergence, it also introduced additional computational burden due to RL training and exhibited sensitivity to hyperparameter settings.

Zhang et al. [24] developed a Q-learning-based Artificial Bee Colony (QABC) algorithm to solve surgery scheduling problems with setup times. Q-learning dynamically guided the selection of six local search operators, improving adaptability to heterogeneous problem structures. Despite its superior results, the method incurs high computational overhead and depends strongly on carefully tuned parameters and operator strategies.

Catteeuw et al. [25] introduced a guided restart mechanism for Hill Climbing (HC), where reinforcement learning partitions the search space and prioritizes promising regions. This improved convergence speed by about 2.3 times compared to random restarts. However, its effectiveness is problem-dependent, as benefits diminish when promising regions are hard to distinguish.

Becerra-Rozas et al. [26] proposed a reinforcement learning-based WOA using SARSA to adaptively select binary encodings during iterations. This method improved accuracy and robustness compared to Q-learning-based variants, but maintaining the RL agent throughout the search increases computational and memory costs, potentially limiting scalability in large-scale optimization tasks.

The integration of RL with metaheuristics presents several common challenges. One significant issue is the computational complexity introduced by maintaining and updating Q-tables for each agent in RL. As the number of agents or the size of the problem increases, this can lead to increased memory requirements and slower execution times, especially for large-scale or high-dimensional optimization problems. Another challenge lies in the sensitivity of hybrid RL-metaheuristic algorithms to hyperparameter tuning. As summarized in Table 1, these methods also face difficulties in scalability and generalization across diverse domains.

The performance of these algorithms heavily depends on the appropriate adjustment of parameters such as learning rates, exploration rates, and the configuration of the metaheuristic component. Incorrect tuning can result in inefficient exploration, premature convergence, or suboptimal solutions. Additionally, the interaction between RL and metaheuristics can sometimes cause instability in the search process, particularly when the exploration–exploitation balance needs to be dynamically adjusted. While the integration of RL has proven beneficial in certain applications, these hybrid approaches often struggle with generalization to different problem domains, indicating a need for further research to improve their scalability and robustness. These challenges highlight the gaps in the current research and suggest that future work could focus on developing methods for automatic parameter tuning, dynamic adjustment of the exploration–exploitation balance, and enhancing the adaptability of these algorithms to a wider range of optimization problems.

The proposed approach focuses on low-level enhancements by optimizing key components of the GWO algorithm. We utilize RL, specifically Q-learning, to select variation operators for exploration and exploitation, as well as to adapt the leader update mechanism. Unlike previous models, where each wolf was treated as an agent with a fixed and gradually decreasing exploration–exploitation rate, our approach treats each wolf as an independent agent. The experiences gathered by wolves is shared in a shared memory, enhancing decision-making and improving the overall algorithmic performance. Furthermore, all changes and improvements are designed in such a way that there is no need for additional parameter tuning and initialization for different problems. The parameter values remain consistent during the analysis and do not require re-adjustment. To the best of our knowledge, no studies were identified in the literature that employed RL to adapt the leader update function. These improvements directly address the key limitations highlighted in Table 1, thereby bridging the identified research gaps.

3. Proposed method

3.1. RL-enhancement framework

Leadership-based metaheuristics, such as GWO, employ a leader-follower dynamic to explore the solution space. In these algorithms, a group of leaders guides followers toward optimal solutions. However, the traditional approach often lacks adaptability, limiting the search efficiency. RL can enhance these algorithms by enabling dynamic and personalized decision-making for both leaders and followers. By learning from interactions with the environment, RL can improve leader selection, following behavior, and leadership strategies, leading to more efficient exploration and faster convergence in optimization tasks. In this paper, we focus on three key perspectives of RL's role in improving leadership-based algorithms:

- **Whom to Follow?** In traditional leadership-based algorithms, followers follow fixed leaders based on a predefined hierarchy. Allowing followers to dynamically select which leader(s) to follow based on their current state or position in the search space can enhance adaptability.
- **How to Follow?** Traditional leader-follower dynamics involve fixed movement patterns for all followers. Instead, a personalized approach can allow each follower to adjust its strategy based on its current state, such as distance from the leader or fitness level.
- **How to Lead?** In traditional leadership-based metaheuristics, leader updates their positions using the same rules as followers. A more flexible leader update process can enable leaders to adjust their strategies based on their performance and feedback from followers.

In this study, we address the aforementioned questions by proposing various RL-based solutions:

- **Dynamic Leader Selection**: Followers are enabled to dynamically choose their leaders based on their current state, with influence coefficients used to evaluate leader effectiveness. This adaptive selection is used to improve the follower's ability to find better solutions by considering real-time leader performance.

Table 1
Hybrid metaheuristic–RL studies and their roles and limitations.

Study (Year)	Algorithm	RL role	Limitations
Yuvaraj et al. (2022) [16]	GWO + RL (RIL)	Task allocation	Computational overhead; sensitivity to hyperparameters; scalability issues
Lin et al. (2021)[17]	RL–GWO (+OCBA)	Parameter tuning	High computational cost; low generalizability; precise tuning required
Emary et al. (2019)[18]	EGWO + RL + ANN	Dynamic parameter update	ANN training overhead; risk of overfitting
Seyyedabbasi et al. (2020)[19]	Q-learning + GWO/WOA	Exploration–exploitation balance	Large Q-table overhead; scalability limitations
Becerra-Rozas (2021)[20]	WOA + SARSA	Strategy selection (binary schemes)	Computational and memory costs; limited scalability
Zhao et al. (2020)[21]	BSO + Q-learning	Operator selection	High computational overhead; parameter dependence
Sadhu et al. (2022)[22]	Firefly + RL	Parameter tuning	Sensitive to parameter settings; limited adaptability
Wu et al. (2020)[23]	PSO + RL (policy net)	Adaptive velocity update	RL training overhead; hyperparameter sensitivity
Zhang et al. (2021)[24]	ABC + Q-learning	Operator selection	Computational cost; operator tuning sensitivity
Catteeuw et al. (2019)[25]	Hill Climbing + RL	Guided restarts	Problem-dependent effectiveness
Becerra-Rozas et al. (2021)[26]	WOA + SARSA	Adaptive encoding selection	RL agent overhead; scalability concerns

Notes. GWO: Grey Wolf Optimizer; WOA: Whale Optimization Algorithm; BSO: Brain Storm Optimization; PSO: Particle Swarm Optimization; ABC: Artificial Bee Colony; ANN: Artificial Neural Network; FJSP: Flexible Job Shop Problem.

- **Personalized Explore-Exploit Trade-off**: To enable adaptive decisions on exploration versus exploitation, RL with shared knowledge among agents was employed. By exchanging states and actions, better-informed decisions are made, allowing for an effective balance based on the specific problem's needs. This collective knowledge sharing enables each follower to assess whether to prioritize exploration or exploitation, fostering a more personalized and effective search process.
- **Adaptive Leader Position Updates**: A novel method was proposed in which leaders use a separate, RL-based mechanism for updating their positions. This allows for adaptive refinement of strategies, making the leaders more effective in guiding followers. This adaptive approach enables strategies to be refined, followers to be guided more effectively, and both exploration and convergence to be optimized, ultimately enhancing the algorithm's overall performance.

These methods are modular, allowing users to choose and apply each based on the specific needs of their optimization problem. Together, they address the limitations of traditional leadership-based metaheuristics and improve their performance.

3.2. Q-learning overview

Q-learning is a widely used RL algorithm designed to help agents learn optimal policies in uncertain and dynamic environments. It belongs to the category of off-policy learning methods, where the agent learns the value of actions independently of the current policy being followed. Q-learning uses a Q-table to store Q-values, which represent the expected long-term rewards for each state-action pair. The algorithm iteratively updates these Q-values by interacting with the environment, observing the immediate rewards, and estimating future rewards based on the Bellman equation. This allows Q-learning to converge to the optimal policy over time, even in the absence of a predefined model of the environment.

Building on this, action selection in Q-learning is commonly guided by an ϵ-greedy strategy, where the agent primarily selects the action with the highest Q-value while occasionally exploring random actions to balance exploration and exploitation.

In this study, Q-learning was selected as the reinforcement learning component due to its simplicity, computational efficiency, and suitability for integration into lightweight metaheuristic frameworks. Metaheuristic algorithms are widely adopted for their low computational cost and rapid convergence properties. Therefore, to preserve these advantages while introducing intelligent, learning-based decision-making, it was essential to select a reinforcement learning method that imposes minimal additional computational overhead.

Compared to more complex alternatives such as deep reinforcement learning, Q-learning offers a significantly lighter implementation while still enabling adaptive behavior based on accumulated experience. Its tabular structure aligns well with the relatively low-dimensional state-action space defined in the proposed framework. Consequently, Q-learning provides a practical and effective balance between performance enhancement and computational feasibility, making it a well-justified choice for the proposed method.

3.3. Original GWO

GWO is a population-based metaheuristic inspired by the social hierarchy and hunting strategies of grey wolves [27]. It categorizes wolves into alpha, beta, delta, and omega, with the top three guiding the hunting process while the others follow. This algorithm is widely used for optimization across various domains due to its simplicity, flexibility, and high performance.

Grey wolves use a strategic hunting technique called encirclement, in which they surround the prey. This behavior is mathematically modeled by Eqs. (1) and (2), where t represents the current iteration, A and C are coefficient vectors, $\vec{X}_p$ denotes the prey's position vector, and $\vec{X}$ represents the position vector of the grey wolf.

$$\vec{D} = \left|\vec{C}\vec{X}_p(t) - \vec{X}(t)\right| \tag{1}$$

$$\vec{X}(t+1) = \vec{X}_p(t) - \vec{A}\cdot\vec{D}. \tag{2}$$

$$\vec{A} = 2\vec{a}\vec{r}_1 - \vec{a} \tag{3}$$

$$\vec{C} = 2\vec{r}_2 \tag{4}$$

$$\vec{a} = 2 - \frac{2 * t}{Iteration_{max}} \tag{5}$$

The variable a decreases linearly from 2 to 0 over the course of iterations. The vectors r_1 and r_2 are randomly generated values from 0 to 1.

In the mathematical model of grey wolf hunting, the three leader wolves-alpha, beta, and delta-are assumed to represent the top solutions, possessing the most information about the optimal solution. The other wolves adjust their positions by averaging those of the leaders, with the final position of each wolf determined using the formula in Eq. (12), promoting convergence toward the optimal solution.

$$\vec{D}_\alpha = \left|C_1\vec{X}_\alpha - \vec{X}\right|, \tag{6}$$

$$\vec{D}_\beta = \left|C_2\vec{X}_\beta - \vec{X}\right|, \tag{7}$$

$$\vec{D}_\delta = \left|C_3\vec{X}_\delta - \vec{X}\right|, \tag{8}$$

$$\vec{X}_1 = \vec{X}_\alpha - A_1\left(\vec{D}_\alpha\right), \tag{9}$$

$$\vec{X}_2 = \vec{X}_\beta - A_2\left(\vec{D}_\beta\right), \tag{10}$$

$$\vec{X}_3 = \vec{X}_\delta - A_3\left(\vec{D}_\delta\right), \tag{11}$$

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3}. \tag{12}$$

3.4. Whom and how to follow?

One of the key limitations of the original GWO is its tendency to get trapped in local optima. This issue arises from the algorithm's inability to maintain sufficient exploration in the final iterations, primarily due to the gradual reduction of the parameter a. As exploration capacity becomes limited, the algorithm struggles to escape local optima in these cases.

To address this challenge, the primary goal is to establish a dynamic balance between exploration and exploitation, personalized for each wolf. Acting as independent agents, wolves autonomously decide whether to explore or exploit based on its current state and shared information from other wolves. This approach enables the algorithm to effectively search for optimal solutions while leveraging existing ones.

To implement this idea, the problem was initially formulated as a Markov Decision Process (MDP). An MDP is defined as a 5-tuple (S, A, P, R, γ), where S is the state space, A is the action space, P is the transition probabilities, R is the reward function, and γ is the discount factor.

3.4.1. State space

The state of an agent is represented as a four-element vector. The first three elements capture the results of the agent's three most recent fitness function evaluations, indicating whether an improvement occurred (1) or not (-1). The fourth element represents the action taken in the previous iteration. This state encapsulates the agent's history. By analyzing this history, the agent can gain insights from past decisions and use them to select its next action, enabling more informed and adaptive behavior. Assuming the optimization problem is minimization, the state of agent i at time t is $State_t^i$, is defined as in Eq. (13):

$$\text{State}_t^i = \left[\text{sign}\left(f_{t-2}^i - f_{t-3}^i\right), \text{sign}\left(f_{t-1}^i - f_{t-2}^i\right), \text{sign}\left(f_t^i - f_{t-1}^i\right), a_{t-1}^i\right]. \tag{13}$$

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x < 0, \\ -1 & \text{if } x \geq 0. \end{cases} \tag{14}$$

where f_t^i represents the fitness value of agent i at time step t, and a_{t-1}^i represents the action of agent i at time step $t-1$.

The proposed approach introduces a shared Q-table as a centralized repository for state-action values, enabling agents to collaboratively learn and make decisions. Agents acting earlier in an iteration update the table with their learned values, allowing subsequent agents to utilize this information in real-time. This dynamic data sharing accelerates convergence by reducing redundant exploration and focusing on promising regions of the search space.

Unlike traditional methods that rely on independent learning and require extensive iterations, the shared Q-table fosters efficient collaboration, improving both short-term and long-term system performance. By continuously exchanging the most recent knowledge, agents adapt to changing conditions, escape local optima, and enhance overall system efficiency with reduced computational overhead.

3.4.2. Action space

The action space for each wolf consists of four possible actions, combining two types of decisions: (1): How to follow? Selecting the parameter a, which governs the wolf's exploration and exploitation behavior, and (2): Whom to follow? Choosing the influence coefficients that determine the relative impact of leader wolves (alpha, beta, and delta) on the current wolf's movement. The goal is to enhance the algorithm's ability to balance exploration and exploitation while escaping local optima. The two actions for balancing a correspond to distinct adjustment functions (a_1 and a_2), while the two actions for influence coefficients represent different following strategies—one where the alpha wolf has a dominant role and another where all leaders contribute equally. The action space includes:

- **Action 1:** Use the adjustment function a_1 for the parameter a, employing a leadership strategy in which the alpha wolf plays a dominant role.
- **Action 2:** Use the adjustment function a_1 for the parameter a, while distributing influence equally among the alpha, beta, and delta wolves.
- **Action 3:** Use the adjustment function a_2 for the parameter a, employing a leadership strategy in which the alpha wolf plays a dominant role.
- **Action 4:** Use the adjustment function a_2 for the parameter a, while distributing influence equally among the alpha, beta, and delta wolves.

This mapping process ensures that each action captures a unique combination of exploration-exploitation adjustment and leadership influence. To further illustrate this process, consider a scenario where a wolf is at a decision point. If the algorithm determines that increased exploration is needed, it may select a_2. If the current optimization state suggests that a dominant leader should be followed, the influence coefficient selection will prioritize the alpha wolf, leading to Action 3. Conversely, if a more balanced influence from all leader wolves is required, Action 4 will be chosen.

A key component of the action space is the dynamic adjustment of the parameter a, which influences the wolves' exploration behavior. Based on the current state:

1. a can be increased significantly if it approaches zero, allowing the algorithm to quickly escape local optima.
2. a can also decrease or increase progressively to balance exploration and exploitation.

Two distinct functions govern the adjustment of a:

$$a_1 = 2 \cdot \cos\left(\frac{\pi}{2}\right) \cdot \left(\frac{l}{Max_{iter}}\right), \tag{15}$$

$$a_2 = \left|2 \cdot \cos\left(\frac{\pi \cdot l}{Max_{iter}}\right)\right|. \tag{16}$$

where l and Max_{iter} represent the current iteration and the maximum number of iterations, respectively. In Eq. (15), a_1 decreases gradually from an initial value 2 to zero as iterations progress. This ensures a transition from an exploration-focused phase in the early stages to an exploitation-focused phase later. In Eq. (16), a_2 varies periodically due to the cosine term, producing oscillatory behavior. The functions a_1 and a_2 are designed to dynamically balance exploration and exploitation during optimization.

To strengthen the leadership hierarchy, the influence coefficients of the leader wolves (alpha, beta, and delta) play a critical role. The alpha wolf's influence is adjusted to have a stronger impact on the wolves' movement direction, increasing the likelihood of finding the optimal solution. Two sets of influence coefficients are defined for this purpose. In the first set of coefficients, $(\phi_1, \phi_2, \phi_3) = (0.6, 0.3, 0.1)$, alpha has the most significant influence, but in the second set of coefficients, all three leader wolves have equal influence, $(\phi_1, \phi_2, \phi_3) = (0.33, 0.33, 0.33)$.

By combining the two adjustment functions for a(a_1 and a_2) and the two influence coefficient sets, the action space for each wolf comprises four possible actions. This design enables each wolf to dynamically adapt its behavior, leveraging both its performance history and real-time feedback. By incorporating these mechanisms, the algorithm enhances exploration, avoids local optima, and efficiently transitions toward the exploitation of promising regions in the search space. This collaborative and adaptive approach ensures robust performance across various optimization scenarios.

3.4.3. Reward

The reward function is defined as shown in Eq. (17). If an agent selects an action that moves it away from a position with better fitness, it receives a negative reward. Conversely, if the agent moves to a position

with improved fitness, it earns a positive reward. This design provides a simple yet effective mechanism for guiding agents towards better solutions.

$$\text{reward}_t^i = f\left(agent_t^i\right) - f\left(agent_{t-1}^i\right) \tag{17}$$

The simplicity of this reward function ensures computational efficiency while providing direct feedback to the agent. By focusing on relative improvement in fitness, the model effectively reinforces beneficial movements without introducing excessive complexity. However, we acknowledge that more sophisticated reward mechanisms could further enhance performance, especially in complex search spaces.

3.4.4. Algorithm

To implement the module, Q-learning has been incorporated to enhance the algorithm's performance while maintaining manageable execution time after applying improvements. In this model, each wolf functions as an independent agent within a multi-agent system, leveraging shared learning for improved coordination and adaptability.

To accelerate learning and utilize shared experiences, all wolves share a common Q-table. This table, continuously updated with the experiences of all agents, enables each wolf to make informed decisions based on collective insights, guiding them toward optimal solutions. The pseudo-code of the proposed algorithm is presented in Algorithm 1.

Algorithm 1 Whom and how to follow.

Input: pop_{size} and max_{iter}
Randomly initialize pop_{size} wolves
Calculate fitness for all wolves
Select α, β, and δ
$t = 1$
for $1 \le t \le Max_{iter}$ **do**
  for $1 \le i \le pop_{size} - 3$ **do**
    Select the action (a_i and (ϕ_1, ϕ_2, ϕ_3)) using shared Q-table and $State_t^i$
    Update the position of $wolf_i$ by using selected action in Eq.(12)
    Calculate the reward using Eq. (17)
    Update the Q-table using the reward and the Q-learning algorithm.
    Update the state vector of $wolf_i$ according to Eq. (13)
  end for
  Update α, β, and δ.
  t++
end for

3.5. How to lead?

In leadership-based algorithms, a key challenge that arises is premature convergence when all leaders update their positions using the same mechanism as the follower agents. Since the update strategy is identical across all agents, the algorithm may converge too quickly to a suboptimal solution. This issue arises because the leaders, intended to guide the search process, lack the necessary behavioral diversity to effectively navigate and explore the search space. Without distinct update strategies for different hierarchy levels, the algorithm lacks the diversity required to escape local optima, resulting in premature convergence and a diminished ability to locate the global optimum.

Using distinct update functions for leaders can be beneficial in addressing this issue. These functions are designed to enable α, β, and δ to explore the search space more effectively, reducing the risk of premature convergence. To implement this idea, the problem was formulated as an MDP as follows.

3.5.1. State space

The state space is defined to reflect the hierarchical structure of leadership-based algorithms. It consists of three distinct states. The first state occurs when the fitness of the alpha wolf improves, as the alpha wolf is the primary leader and plays a crucial role in guiding the search process. The second state is defined for situations where the alpha wolf's fitness does not improve, but at least one of the other leaders shows progress. This ensures that secondary leaders contribute to the decision-making process when the primary leader is stagnant. Finally, the third state captures scenarios where no leader, including the alpha wolf, demonstrates any fitness improvement. This structure allows the algorithm to adapt its strategies dynamically based on the progress of the leaders, enabling better exploration and exploitation of the search space to escape local optima and move toward the global solution.

3.5.2. Action space

The action space includes three actions, each corresponding to a different update function for altering the positions of leaders, with each serving as a different strategy. At each step, an agent can select one of these functions. The three selected functions in this section are designed to maintain an effective balance between exploration and exploitation for leaders.

To find optimal solutions in leadership-based algorithms, the leaders must effectively explore new regions and exploit them to identify the best solutions. Through experimental analysis, two distinct functions have been used to guide the exploration and exploitation of leaders. Additionally, the standard update function from the original GWO is included as an option, which reduces the distance between the leader wolves and the current best solution, refining their search process.

The selection of Gamma and Exponential distributions for exploration and exploitation control was based on their statistical properties, theoretical foundations, and empirical effectiveness. The Gamma distribution provides flexibility in modeling exploration behaviors with varying intensities, allowing for an adaptive search mechanism. Its ability to generate skewed distributions makes it suitable for emphasizing exploration in the early stages and gradually reducing it over time. Conversely, the Exponential distribution ensures smooth exploitation transitions, preventing abrupt changes in decision-making.

Furthermore, we conducted extensive empirical experiments to identify suitable functions for generating exploration and exploitation values. Through these experiments, we evaluated different probabilistic distributions and selected the ones that demonstrated effective balance and convergence behavior in our optimization framework. The action space includes:

- **Standard Exponential Distribution Function:** This function generates small values and implements gradual changes, facilitating controlled and continuous updates to the wolves' positions. Such gradual updates help reduce the likelihood of getting trapped in local optima. The function is defined as in Eq. (18):

$$f(x) = e^{-x}, \quad x \ge 0, \tag{18}$$

  where x is a non-negative random variable sampled from the exponential distribution. This distribution is characterized by a high probability of generating small values while occasionally producing larger values, which supports an adaptive exploration mechanism.
- **Gamma Distribution Function:** Due to its flexibility and adjustable parameters, this function can balance exploration and exploitation effectively. The gamma distribution allows wolves to make gradual position changes using small parameter values. At the same time, it can support larger changes, aiding in escaping local optima when necessary. The function is defined as in Eq. (19):

$$f(x; k, \theta) = \frac{x^{k-1} e^{-x/\theta}}{\theta^k \Gamma(k)}, \quad x > 0,\ k > 0,\ \theta > 0 \tag{19}$$

  where x represents the random variable, and k is the shape parameter that controls the distribution's form, determining its skewness

and tail behavior. The shape parameter k in this case is 1.5, and the scale parameter θ is 0.4, which determines the spread of the distribution, influencing how widely the generated values are distributed. Additionally, $\Gamma(k)$ refers to the Gamma function, which is defined as in Eq. (20):

$$\Gamma(k) = \int_0^\infty t^{k-1} e^{-t}\, dt \tag{20}$$

$\Gamma(k)$ generalizes the factorial function, and when k is a positive integer, it reduces to $\Gamma(k) = (k-1)!$. For non-integer values of k, $\Gamma(k)$ is computed using the integral definition provided above.

- **Standard GWO Update Function:** This is the traditional position update function in GWO, which guides wolves based on the current best solutions as in Eq. (12).

The Gamma and standard exponential functions play different roles in the update process. The standard exponential function is used for exploration, while the Gamma function is used for exploitation. The updated position is calculated as follows in Eq. (21):

$$\vec{X}(t+1) = \vec{X} \pm \vec{X} \times r \tag{21}$$

The parameter r is generated by actions and is utilized to update the wolves' positions. Based on their current positions, they decide whether to explore or exploit.

3.5.3. Reward

The reward function is designed based on improvements in the positions of the leader wolves. If the chosen action results in an improvement in a leader's position, the reward is assigned according to the leader's significance in the hierarchy. Specifically, the alpha wolf receives the highest reward, followed by the beta and delta wolves. The total reward is the sum of the rewards for all leaders that show improvement. For example, if only the alpha and delta wolves improve, the final reward will be the sum of the rewards for these two leaders.

Mathematically, let $\Delta x_\alpha, \Delta x_\beta, \Delta x_\delta$ represent the improvements in the fitness of the alpha, beta, and delta wolves, respectively. The reward for each leader is defined in Eqs. (22)–(24):

$$R_\alpha = \begin{cases} r_\alpha & \text{if } \Delta x_\alpha > 0, \\ 0 & \text{if } \Delta x_\alpha \le 0, \end{cases} \tag{22}$$

$$R_\beta = \begin{cases} r_\beta & \text{if } \Delta x_\beta > 0, \\ 0 & \text{if } \Delta x_\beta \le 0, \end{cases} \tag{23}$$

$$R_\delta = \begin{cases} r_\delta & \text{if } \Delta x_\delta > 0, \\ 0 & \text{if } \Delta x_\delta \le 0. \end{cases} \tag{24}$$

where $r_\alpha > r_\beta > r_\delta$ reflects the relative significance of each leader, with the alpha wolf receiving the highest reward. The reward values are set as 30 for the alpha wolf, 15 for the beta wolf, and 5 for the delta wolf, determined through trial and error. These values are adjusted based on the importance of each leader, as an action that guides the alpha wolf to a better position should receive a higher reward.

The total reward R_{total} is the sum of the individual rewards for the leaders that demonstrate improvement as shown in Eq. (25):

$$R_{\text{total}} = R_\alpha + R_\beta + R_\delta \tag{25}$$

If no improvement occurs for any of the leaders, the total reward is negative:

$$R_{\text{total}} = -1 \quad \text{if} \quad \Delta x_\alpha \le 0, \Delta x_\beta \le 0, \Delta x_\delta \le 0. \tag{26}$$

3.5.4. Algorithm

The Q-learning algorithm is employed to implement this module too. Based on the current status of the leader wolves, the agent selects the optimal update function to enhance performance in the search space. The pseudocode of the proposed algorithm is presented in Algorithm 2.

Algorithm 2 How to lead.

1: Input: max_{iter}.
2: Randomly initialize pop_{size} wolves
3: Calculate fitness for all wolves
4: Select α, β, and δ
5: $t = 1$
6: **for** $1 \le t \le max_{iter}$ **do**
7: Select the action (update function f) using shared Q-table and $State_t^i$
8: Update the position of all leaders by using selected action in Eq. (21)
9: Calculate reward using Eq. (25)
10: Update the Q-table using the reward and the Q-learning algorithm.
11: Update the state vector of leaders
12: **end for**
13: Update α,β,δ
14: t++

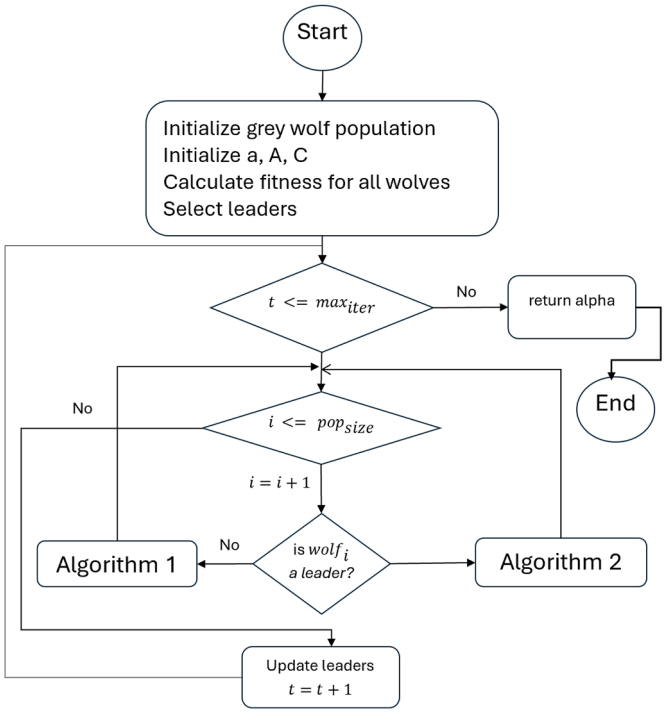


Fig. 1. Flowchart of RL-LGWO.

3.6. RL-LGWO

The flowchart of the RL-LGWO algorithm, depicted in Fig. 1, illustrates its overall structure and workflow, which is built upon the integration of two previously defined algorithms, Algorithms 1 and 2. Note that Algorithm 1 is executed only once in each iteration and updates all leaders during its execution. The flowchart visually represents the interaction between these components, detailing the sequential and iterative steps of the algorithm. It highlights the initialization phase, leader position updates, non-leader position updates, and the evaluation of solutions.

4. Experimental results and analysis

4.1. Experimental setup

The proposed algorithm, RL-LGWO, was compared with eight previous works including two GWO-enhancing algorithms (AGWO [30], and EGWO [31]), three RL-based GWO variants (RLGWO [32], QGWO [33],

Table 2
Parameter settings of the compared algorithms.

Algorithm	Parameter settings
GWO	Control parameter $a: 2 \rightarrow 0$ [33]
AGWO	$\epsilon = 10^{-5}$, $\gamma = 0.95$, minimum steps $= 10$, threshold $= 10^{-3}$ [30]
EGWO	Penalty factor $= 0.01$, control constant $= 2$ [31]
QGWO	$\lambda = 0.1$, $\gamma = 0.9$ [33]
RLEGWO	Penalty factor $= 0.01$, control constant $= 2$, $\lambda = 0.9$, $\gamma = 0.5$ [34]
RLGWO	$o_1 = 0.6$, $o_2 = 0.3$, $o_3 = 0.1$, learning rate $\alpha_{\text{final}} = 0.1$, discount factor $\gamma = 0.5$, punishment $p_{\text{init}} = -10$, $p_{\text{final}} = -2$ [32]
PSO	$V_{\max} = 6$, $w_{\max} = 0.9$, $w_{\min} = 0.2$, $c_1 = c_2 = 2$ [28]
WOA	$a: 2 \rightarrow 0$, $a_2: -1 \rightarrow -2$, $b = 1$, $p \in [0, 1]$ [29]
Proposed (RL_LGWO)	Learning rate $= 0.01$, discount factor $= 0.95$, decay $= 0.995$ (this study)

and RLEGWO [34]), PSO [28], WOA [29], and original GWO. A diverse set of benchmark algorithms was carefully selected to ensure fairness and comprehensive evaluation. This selection comprises widely used metaheuristic algorithms, reinforcement learning-enhanced variants of the GWO, and conventional improvements of GWO without reinforcement learning. Such diversity promotes a balanced comparison framework and strengthens the validity of the experimental results, offering a robust basis for demonstrating the proposed method's effectiveness and superiority. The source code of our algorithm is publicly available in Roayaei [35].

All experiments were conducted under identical conditions to ensure a fair comparison among the algorithms. Each algorithm was executed with a population size of 30 over 500 fixed iterations. To address the stochastic nature of metaheuristic algorithms, each experiment was repeated 20 independent times, and the mean and standard deviation of the results were reported. All algorithms were implemented in Python 3.12, and the experiments were performed on a MacBook Air with an Apple M1 processor and 8 GB of RAM.

4.2. Parameter settings

The control parameters of each algorithm were set according to their original papers or commonly accepted values in the literature, as summarized in Table 2. For the proposed algorithm, the learning rate was set to $\alpha = 0.01$, the discount factor to $\gamma = 0.95$, and the decay factor to $d = 0.995$. These values were selected after preliminary experiments and sensitivity analysis to ensure a balance between exploration and exploitation.

4.3. Benchmark test functions

A diverse set of 23 CEC benchmark functions was utilized to evaluate the proposed algorithm, covering both unimodal and multi-modal search spaces to rigorously assess its ability to identify global minima. The performance was measured based on different factors, including computational efficiency, solution accuracy, stability, and convergence. The benchmark suite comprises 7 unimodal functions (see Fig. 2), 6 multi-modal functions (see Fig. 3), and 10 multi-modal functions with fixed dimensions (see Fig. 4). To ensure a comprehensive evaluation of the proposed algorithm, each algorithm was executed 20 times on each benchmark function. The mean and standard deviation of the final solutions were then computed. Fig. 5 illustrates the 2D versions of the benchmark functions used.

In addition, the CEC'22 benchmark suite, which consists of 12 challenging shifted, rotated, hybrid, and composition functions [36], was also employed to further validate the robustness of the proposed algorithm. For these functions, only the mean and standard deviation of the results over multiple runs were reported (see [Table]3).

4.4. Accuracy analysis

The proposed algorithm has been compared with other algorithms on benchmark functions, and the results are presented in Tables 4 and 5.

Table 3
Details of the 12 test functions in the CEC'22 benchmark suite [36].

Category	Function	Limits	Global optimum
Basic	F1: Shifted and Rotated Zakharov	[−100,100]	300
	F2: Shifted and Rotated Rosenbrock	[−100,100]	400
	F3: Shifted and Rotated Schaffer's F6	[−100,100]	600
	F4: Shifted and Rotated Non-cont. Rastrigin	[−100,100]	800
	F5: Shifted and Rotated Levy	[−100,100]	900
Hybrid	F6: Hybrid Function 1 ($N = 3$)	[−100,100]	1800
	F7: Hybrid Function 2 ($N = 6$)	[−100,100]	2000
	F8: Hybrid Function 3 ($N = 5$)	[−100,100]	2200
Composite	F9: Composition Function 1 ($N = 5$)	[−100,100]	2300
	F10: Composition Function 2 ($N = 4$)	[−100,100]	2400
	F11: Composition Function 3 ($N = 5$)	[−100,100]	2600
	F12: Composition Function 4 ($N = 6$)	[−100,100]	2700

The proposed algorithm consistently achieves the lowest mean error (Mean) and standard deviation (Std) across 17 benchmark functions compared to other algorithms, demonstrating high accuracy and stability. In addition to the classical CEC benchmark suite, the proposed algorithm was also evaluated on the CEC'22 benchmark functions [36], which consist of five basic, three hybrid, and four composition functions (see Table 3). For these experiments, the mean and standard deviation of the solutions obtained over 20 independent runs were computed. Each algorithm was executed with a population size of 5 over 200 iterations, and the results are summarized in Table 6.

Specifically, RL-LGWO achieves near-zero mean values and minimal standard deviations on F1 and F2, indicating its strong ability to converge to the global optimum without being trapped in local optima. On function F3, RL-LGWO also provides a significantly lower mean than other algorithms, although the improvement over competitors is less pronounced in this case. For more challenging functions such as F12 and F21, RL-LGWO attains favorable mean and standard deviation values, again outperforming other algorithms overall. These results highlight its superior ability to navigate complex search spaces and maintain robustness against local optima.

Across the benchmark suite, RL-LGWO consistently demonstrates a remarkable ability to minimize both the mean and standard deviation, particularly on highly complex and high-dimensional functions such as F1, F2, F4, F9, and F10, where it achieves clear superiority over most existing algorithms. Nevertheless, there are specific cases where other methods show competitive or better performance. For instance, on F6, PSO achieves the best mean result, which can be attributed to its strong exploitative dynamics in narrow landscapes; RL-LGWO, by maintaining a higher exploration pressure, produces competitive but not optimal results on this function. On F7, GWO slightly outperforms RL-LGWO, benefiting from its symmetric encircling mechanism in relatively smooth multimodal landscapes. Similarly, in F8, WOA provides the lowest mean due to its spiral-based exploitation strategy, which is particularly effective in irregular rugged basins. In these cases, RL-LGWO trades slight reductions in best-case accuracy for improved stability across runs.

RL-LGWO achieves the optimal value of zero on F9 and F11 (with RLGWO also reaching zero on F11), and it consistently outperforms

Function	Dim	Range	f_{min}
$f_1(x) = \sum_{i=1}^{n} x_i^2$	30	[-100,100]	0
$f_2(x) = \sum_{i=1}^{n} \lvert x_i \rvert + \prod_{i=1}^{n} \lvert x_i \rvert$	30	[-10,10]	0
$f_3(x) = \sum_{i=1}^{n} \left(\sum_{j-1}^{i} x_j\right)^2$	30	[-100,100]	0
$f_4(x) = \max_i \{\lvert x_i \rvert, 1 \le i \le n\}$	30	[-100,100]	0
$f_5(x) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	30	[-30,30]	0
$f_6(x) = \sum_{i=1}^{n} ([x_i + 0.5])^2$	30	[-100,100]	0
$f_7(x) = \sum_{i=1}^{n} i x_i^4 + random[0,1)$	30	[-1.28,1.28]	0

Fig. 2. Unimodal benchmark functions.

Function	Dim	Range	f_{min}
$F_8(x) = \sum_{i=1}^{n} -x_i sin\left(\sqrt{\lvert x_i \rvert}\right)$	30	[-500,500]	-418.9829×5
$F_9(x) = \sum_{i=1}^{n} [x_i^2 - 10cos(2\pi x_i) + 10]$	30	[-5.12,5.12]	0
$F_{10}(x) = -20exp\left(-0.2\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}\right) - exp\left(\frac{1}{n}\sum_{i=1}^{n} cos(2\pi x_i)\right) + 20 + e$	30	[-32,32]	0
$F_{11}(x) = \frac{1}{4000}\sum_{i=1}^{n} x_i^2 - \prod_{i=1}^{n} cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$	30	[-600,600]	0
$F_{12}(x) = \frac{\pi}{n}\{10sin(\pi y_1) + \sum_{i=1}^{n-1}(y_i - 1)^2[1 + 10sin^2(\pi y_{i+1})] + (y_n - 1)^2\} + \sum_{i=1}^{n} u(x_i, 10,100,4)$ $y_i = 1 + \frac{x_i+1}{4}$ $u(x_i, a, k, m) = \begin{cases} k(x_i - a)^m & x_i > a \\ 0 & -a < x_i < a \\ k(-x_i - a)^m & x_i < -a \end{cases}$	30	[-50,50]	0
$F_{13}(x) = 0.1\{sin^2(3\pi x_1) + \sum_{i=1}^{n}(x_i - 1)^2[1 + sin^2(3\pi x_i + 1)] + (x_n - 1)^2[1 + sin^2(2\pi x_n)]\} + \sum_{i=1}^{n} u(x_i, 5,100,4)$	30	[-50,50]	0

Fig. 3. Multi-modal benchmark functions.

Function	Dim	Range	f_{min}
$F_{14}(x) = \left(\frac{1}{500} + \sum_{j=1}^{25} \frac{1}{j + \sum_{i=1}^{2}(x_i - a_{ij})^6}\right)^{-1}$	2	[-65,65]	1
$F_{15}(x) = \sum_{i=1}^{11} \left[a_i - \frac{x_1(b_i^2 + b_i x_2)}{b_i^2 + b_i x_3 + x_4}\right]^2$	4	[-5,5]	0.00030
$F_{16}(x) = 4x_1^2 - 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1x_2 - 4x_2^2 + 4x_2^4$	2	[-5,5]	-1.0316
$F_{17}(x) = \left(x_2 - \frac{5.1}{4\pi^2}x_1^2 + \frac{5}{\pi}x_1 - 6\right)^2 + 10\left(1 - \frac{1}{8\pi}\right)cosx_1 + 10$	2	[-5,5]	0.398
$F_{18}(x) = [1 + (x_1 + x_2 + 1)^2(19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2)] \times [30 + (2x_1 - 3x_2)^2 \times (18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2)]$	2	[-2,2]	3
$F_{19}(x) = -\sum_{i=1}^{4} c_i exp\left(-\sum_{j=1}^{3} a_{ij}(x_j - p_{ij})^2\right)$	3	[1,3]	-3.86
$F_{20}(x) = -\sum_{i=1}^{4} c_i exp\left(-\sum_{j=1}^{6} a_{ij}(x_j - p_{ij})^2\right)$	6	[0,1]	-3.32
$F_{21}(x) = -\sum_{i=1}^{5}[(X - a_i)(X - a_i)^T + c_i]^{-1}$	4	[0,10]	-10.1532
$F_{22}(x) = -\sum_{i=1}^{7}[(X - a_i)(X - a_i)^T + c_i]^{-1}$	4	[0,10]	-10.4028
$F_{23}(x) = -\sum_{i=1}^{10}[(X - a_i)(X - a_i)^T + c_i]^{-1}$	4	[0,10]	-10.5363

Fig. 4. Fixed-dimension mult-imodal benchmark functions.

competitors on F20–F23. For F16–F19, most algorithms, including RL-LGWO, converge to the known optima, confirming the effectiveness of the proposed design in unimodal and fixed-dimension multimodal functions. These results indicate that the reinforcement learning mechanism embedded in the leadership update process effectively enhances exploration–exploitation balance, prevents premature convergence, and ensures reliable performance across diverse landscapes.

Although RL-LGWO achieves superior accuracy on most benchmark functions, it is not universally optimal. This limitation can be explained by two factors. First, according to the no-free-lunch theorem, no

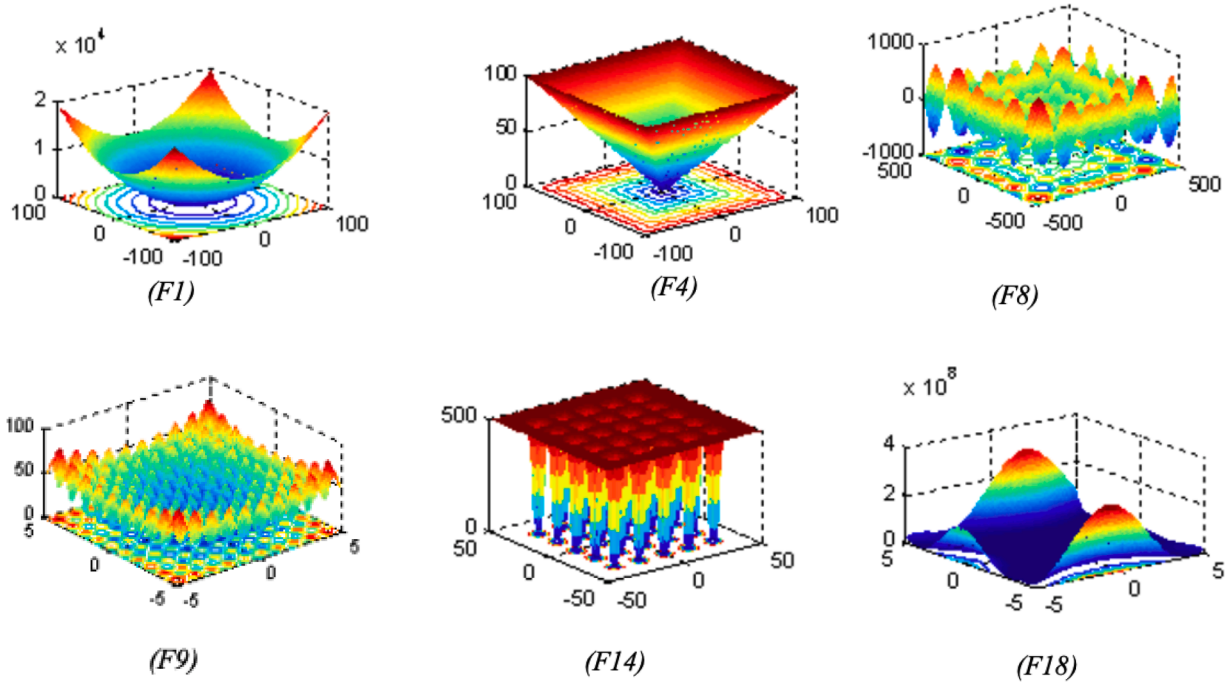


Fig. 5. 2-D versions of benchmark functions.

Table 4
Results of CEC classical benchmark functions (F1–F12).

F	Metric	Proposed method	AGWO	EGWO	QGWO	RLEGWO	RLGWO	GWO	PSO	WOA
F1	Mean	**3.92E−105**	1.25E−09	6.19E−03	2.82E−55	1.59E−46	1.02E−28	7.00E−31	1.52E−04	4.57E−75
	Std	1.75E−104	2.34E−09	5.68E−03	2.88E−55	5.16E−46	4.48E−28	1.12E−30	2.04E−04	1.67E−74
F2	Mean	**8.20E−59**	1.20E−07	4.04E+00	1.55E−33	2.05E−27	1.30E−20	1.14E−18	5.52E+00	3.69E−51
	Std	2.84E−58	3.23E−07	6.79E+00	3.10E−33	2.96E−27	3.40E−20	1.01E−18	7.59E+01	1.12E−50
F3	Mean	**1.32E−32**	9.02E−04	2.46E+03	1.34E−03	3.30E−05	4.14E−13	6.38E−05	8.68E+01	4.45E+04
	Std	1.77E−31	4.86E−04	2.17E+03	5.98E−03	8.14E−05	1.70E−12	1.60E−04	2.76E+01	1.61E+04
F4	Mean	**5.49E−60**	5.60E−07	7.17E+00	1.57E−12	3.09E−11	6.46E−06	2.98E−06	1.11E+00	4.59E+01
	Std	2.26E−59	6.23E−07	2.04E+00	1.57E−12	5.30E−11	1.39E−05	3.31E−06	2.05E−01	2.50E+01
F5	Mean	**2.66E+01**	2.67E+01	1.22E+02	**2.66E+01**	2.72E+01	3.02E+01	2.68E+01	6.40E+01	2.79E+01
	Std	8.79E−01	4.91E−01	7.93E+01	3.93E−01	6.65E−01	1.65E+01	7.76E−01	4.01E+01	4.99E−01
F6	Mean	9.82E−01	6.41E−01	9.55E+00	3.90E−01	9.09E+00	9.85E−01	7.93E−01	**1.71E−04**	4.11E−01
	Std	3.54E−01	2.71E−01	1.31E−02	3.62E−01	2.63E−01	9.85E−01	4.51E−01	2.11E−04	2.59E−01
F7	Mean	1.67E−03	6.92E−03	8.78E−02	3.71E−03	1.79E−03	3.27E−01	**1.14E−03**	5.90E+00	2.70E−03
	Std	1.05E−03	6.41E−03	6.31E−02	8.21E−04	3.71E−03	3.27E−01	1.14E−03	5.90E+00	2.70E−03
F8	Mean	−5.76E+03	−5.35E+03	−5.15E+48	−3.54E+03	−3.35E+03	−5.44E+03	−6.04E+03	−4.77E+03	**−9.36E+03**
	Std	1.23E+03	2.08E+03	1.95E+49	3.57E+02	2.43E+02	2.52E+03	2.52E+02	2.52E+03	1.48E+03
F9	Mean	**0.00E+00**	1.30E+01	4.76E+01	5.56E+01	1.99E+14	4.01E+01	1.12E+02	0.00E+00	0.00E+00
	Std	0.00E+00	1.34E+01	1.97E+01	1.61E+01	2.78E+14	1.77E+01	1.30E+01	0.00E+00	0.00E+00
F10	Mean	**1.33E−15**	2.29E−07	1.49E−01	1.25E−14	1.13E−14	3.34E+01	2.66E−02	4.00E−15	3.69E−11
	Std	1.51E−15	6.90E−08	2.95E−01	2.92E−15	2.93E−15	4.09E−02	5.79E−15	2.31E−15	1.65E−10
F11	Mean	**0.00E+00**	6.60E−03	6.65E−02	6.13E−04	1.32E−03	**0.00E+00**	1.00E−02	1.84E−02	0.00E+00
	Std	0.00E+00	9.60E−03	8.69E−02	2.74E−03	4.07E−03	0.00E+00	8.44E−03	9.03E−03	0.00E+00
F12	Mean	5.72E−02	3.56E−02	3.85E−02	8.26E−02	8.85E−02	2.66E−01	3.88E−02	**5.19E−03**	1.86E−02
	Std	2.75E−02	2.15E−02	1.03E−02	2.77E−02	2.49E−02	6.26E−01	2.13E−02	2.32E−02	1.35E−02

metaheuristic can guarantee the best performance across all optimization problems, as different problem landscapes favor different search dynamics. Second, the design of RL-LGWO emphasizes maintaining a stronger exploration ability to avoid premature convergence. While this improves robustness and stability, in some benchmark cases (e.g., functions with narrow or smooth landscapes), algorithms with stronger exploitative mechanisms such as PSO or WOA can reach slightly better results. Therefore, RL-LGWO trades off a small loss in best-case accuracy for enhanced generality and reliability across diverse functions.

The results presented in Table 6 clearly demonstrate the superiority of the proposed algorithm across the majority of CEC'22 benchmark functions. In particular, the algorithm consistently achieves the lowest mean error and competitive standard deviations on functions such as F1, F2, F3, F4, and F5, highlighting its effectiveness in solving unimodal

Table 5
Results of CEC classical benchmark functions (F13–F23).

F	Metric	Proposed Method	AGWO	EGWO	QGWO	RLEGWO	RLGWO	GWO	PSO	WOA
F13	Mean	8.18E−01	5.34E−01	1.65E+00	8.91E−01	9.61E−01	2.75E+01	6.21E−01	**6.08E−03**	5.49E−01
	Std	2.12E−01	2.33E−01	1.64E+00	2.56E−01	1.93E−01	1.10E+02	2.19E−01	1.25E−02	1.83E−01
F14	Mean	4.62E+00	5.48E+00	**1.89E+00**	5.01E+00	2.67E+00	8.68E+00	4.79E+00	2.90E+00	2.91E+00
	Std	4.21E+00	5.23E+00	1.60E+00	4.41E+00	2.92E+00	4.11E+00	5.37E+00	5.30E−03	7.44E−04
F15	Mean	**4.17E−04**	4.45E−03	1.41E−03	4.68E−03	5.50E−03	1.24E−03	8.05E−03	7.90E−03	5.64E−04
	Std	2.07E−04	8.39E−03	1.97E−03	8.05E−03	8.05E−03	1.77E−03	8.88E−03	7.90E−03	5.64E−04
F16	Mean	**−1.03E+00**	**−1.03E+00**	**−1.03E+00**	**−1.03E+00**	**−1.03E+00**	**−1.03E+00**	**−1.03E+00**	**-1.03E+00**	**−1.03E+00**
	Std	1.95E−08	1.00E−10	0.00E+00	9.53E−06	1.07E−05	6.91E−04	1.37E−08	0.00E+00	1.38E−09
F17	Mean	**3.98E−01**	**3.98E−01**	**3.98E−01**	**3.98E−01**	**3.98E−01**	4.28E−01	**3.98E−01**	**3.98E−01**	**3.98E−01**
	Std	2.53E−05	1.61E−04	0.00E+00	1.15E−03	4.84E−04	3.06E−05	4.05E−06	0.00E+00	7.83E−06
F18	Mean	**3.00E+00**	**3.00E+00**	**3.00E+00**	**3.00E+00**	**3.00E+00**	2.45E+01	**3.00E+00**	**3.00E+00**	**3.00E+00**
	Std	1.61E−05	6.71E−01	2.06E−15	2.05E−05	1.55E−05	3.81E+01	3.21E−05	1.41E−15	2.98E−05
F19	Mean	**−3.86E+00**	**−3.86E+00**	**−3.86E+00**	**−3.86E+00**	**−3.86E+00**	−3.69E+00	**−3.86E+00**	**−3.86E+00**	−3.85E+00
	Std	3.15E−03	2.58E−03	1.44E−16	2.98E−03	2.34E−03	6.34E−03	1.59E−03	3.23E−03	2.80E−02
F20	Mean	**−3.26E+00**	−3.24E+00	−3.22E+00	−3.22E+00	−3.20E+00	−3.19E+00	−3.25E+00	−3.13E+00	−3.19E+00
	Std	8.64E−02	7.32E−02	5.28E−02	5.97E−02	1.17E−01	9.56E−02	8.40E−02	3.52E−01	1.38E−01
F21	Mean	**−8.95E+00**	−8.26E+00	−4.63E+00	−6.36E+00	−4.86E+00	−4.43E+00	−8.89E+00	−7.38E+00	−7.10E+00
	Std	2.84E+00	3.56E+00	2.93E+00	1.12E+00	1.96E+00	1.96E+00	2.65E+00	2.93E+00	2.91E+00
F22	Mean	**−1.01E+01**	−1.00E+01	−7.58E+00	−6.80E+00	−6.66E+00	−6.08E+00	−9.78E+00	−8.16E+00	−7.08E+00
	Std	1.19E+00	1.71E+00	3.27E+00	1.46E+00	1.36E+00	3.26E+00	1.00E+00	3.26E+00	3.13E+00
F23	Mean	**−1.05E+01**	−9.60E+00	−6.85E+00	−6.22E+00	−6.15E+00	−4.39E+00	**−1.05E+01**	−8.03E+00	−7.95E+00
	Std	1.23E+03	2.90E+00	3.81E+00	1.41E+00	1.36E+00	2.15E+00	1.07E+00	3.57E+00	2.96E+00

Table 6
Results of CEC'22 benchmark functions (F1–F12).

F	Metric	Proposed Method	AGWO	EGWO	QGWO	RLEGWO	RLGWO	GWO	PSO	WOA
F1	Mean	**8.09E+03**	1.78E+04	2.93E+04	1.94E+04	1.66E+04	1.69E+04	1.86E+04	1.85E+04	3.90E+04
	Std	5.12E+03	8.45E+03	1.46E+04	9.94E+03	9.17E+03	7.16E+03	1.18E+04	1.30E+04	1.41E+04
F2	Mean	**5.76E+05**	2.99E+07	3.88E+07	8.29E+07	1.56E+07	9.41E+08	4.89E+07	1.30E+07	1.75E+09
	Std	1.14E+06	6.05E+07	5.32E+07	1.55E+08	1.21E+07	1.04E+09	1.32E+08	2.50E+07	1.33E+09
F3	Mean	**3.54E+00**	3.87E+00	4.00E+00	3.96E+00	4.00E+00	4.17E+00	3.98E+00	3.95E+00	4.05E+00
	Std	2.94E−01	4.20E−01	2.83E−01	2.83E−01	2.73E−01	2.07E−01	2.51E−01	2.84E−01	2.73E−01
F4	Mean	**2.81E+02**	1.30E+03	2.32E+03	1.44E+03	1.09E+03	4.27E+03	1.07E+03	3.85E+02	4.64E+03
	Std	1.74E+02	1.59E+03	2.06E+03	6.37E+02	5.59E+02	2.23E+03	8.12E+02	4.79E+02	2.06E+03
F5	Mean	**2.26E+02**	2.46E+02	7.01E+02	2.46E+02	3.96E+02	8.18E+02	5.06E+02	8.60E+02	1.17E+03
	Std	1.41E+02	2.83E+02	5.86E+02	1.98E+02	6.10E+02	3.19E+02	4.78E+02	2.99E+02	7.65E+02
F6	Mean	7.83E+05	3.70E+06	5.24E+06	2.82E+06	2.02E+06	8.01E+06	2.05E+06	**5.78E+05**	2.02E+07
	Std	4.60E+05	7.07E+06	1.07E+07	3.35E+06	1.85E+06	6.02E+06	2.48E+06	5.17E+05	3.25E+07
F7	Mean	**1.89E+04**	3.55E+04	2.58E+04	2.85E+04	4.79E+04	5.88E+08	3.29E+04	1.90E+04	5.39E+08
	Std	9.33E+03	1.78E+04	1.57E+04	1.67E+04	2.21E+04	9.36E+08	2.27E+04	9.10E+03	1.71E+09
F8	Mean	1.41E+04	1.90E+04	1.21E+04	4.33E+04	4.71E+04	2.12E+04	1.75E+04	**9.99E+03**	2.96E+04
	Std	9.97E+03	1.41E+04	1.10E+04	4.04E+04	6.78E+04	9.33E+03	2.24E+04	9.79E+03	2.34E+04
F9	Mean	**4.26E+08**	9.20E+08	5.12E+08	7.76E+08	5.35E+08	1.22E+09	8.98E+08	1.19E+09	9.47E+08
	Std	8.05E+07	3.29E+08	2.45E+08	4.38E+08	1.77E+08	5.01E+08	4.84E+08	4.38E+08	4.50E+08
F10	Mean	**1.15E+09**	1.47E+09	1.60E+09	1.44E+09	1.81E+09	3.68E+09	1.70E+09	1.99E+09	5.05E+09
	Std	5.76E+08	8.67E+08	1.21E+09	8.62E+08	8.80E+08	1.82E+09	9.98E+08	1.36E+09	1.95E+09
F11	Mean	**4.31E+06**	2.50E+08	1.93E+08	1.26E+08	2.73E+08	6.74E+08	1.85E+08	4.21E+08	1.24E+09
	Std	7.09E+06	4.18E+08	4.29E+08	2.17E+08	3.05E+08	4.90E+08	2.35E+08	9.48E+08	1.10E+09
F12	Mean	**1.75E+08**	3.87E+08	2.16E+08	3.73E+08	3.96E+08	1.47E+09	3.68E+08	4.39E+08	6.90E+08
	Std	1.99E+08	1.74E+08	8.21E+07	2.10E+08	2.92E+08	1.34E+09	1.88E+08	2.15E+08	2.88E+08

and moderately multimodal problems. Moreover, for highly complex functions like F9, F10, F11, and F12, the proposed method significantly outperforms the competing algorithms by achieving lower mean values and improved stability, confirming its robustness in handling large-scale multimodal landscapes. These results indicate that the reinforcement learning mechanism embedded in the leadership update process successfully enhances the balance between exploration and exploitation, allowing the algorithm to converge toward global optima while avoiding premature convergence.

Nevertheless, there are a few instances where the proposed algorithm does not yield the best performance. For example, in F6 and F8, the hybrid nature of the functions introduces subcomponents with highly diverse characteristics (e.g., Bent Cigar, Katsuura, Schwefel), which creates challenging landscapes with narrow valleys and discontinuities. In

such cases, algorithms with stronger exploitative tendencies, such as PSO, occasionally achieve slightly better results. Despite this, the proposed method still produces competitive outcomes, remaining among the top-performing algorithms.

Overall, the experimental results on the CEC'22 suite confirm that the proposed algorithm provides superior accuracy, robustness, and stability in most scenarios, particularly in complex multimodal environments. The minor performance gaps observed in specific hybrid functions highlight potential directions for future research, such as further enhancing adaptive parameter control to better handle functions with highly heterogeneous landscapes.

4.5. Ablation study

An ablation study was conducted to quantify the individual contributions of the proposed modifications. Four variants were evaluated: the baseline GWO; GWO augmented solely with the exploration–exploitation balancing component (PM-Exp); GWO augmented solely with the leader-update enhancement (PM-Update); and the integrated Proposed Method combining both components. All algorithmic variants were evaluated under the similar experimental settings and parameter configurations, as described in the preceding section, with mean fitness values computed for the complete set of benchmark functions (F1–F23).

Table 7 provides the mean results for each benchmark function. The PM-Update variant achieves the best result on the largest number of functions and attains the lowest average rank, indicating that refinements to the leader-update mechanism are the principal contributor to the observed performance gains. By contrast, PM-EXP predominantly enhances exploration and thereby shows advantages on problems that require broad search behavior. The fully integrated Proposed Method (RL-LGWO) exhibits robust and competitive performance across all function categories, which suggests complementarity between the two modifications: PM-Exp preserves or increases population diversity during early search stages, while PM-Update strengthens leader-driven exploitation to accelerate and stabilize convergence in later stages.

The PM-Update component refines the leader-update formula, thereby directly modifying the manner in which information is transmitted from leaders to followers. As a result, once a promising region is identified, leaders guide the population toward that region with greater precision and stability. This improved guidance concentrates the search more effectively in smooth basins and other landscapes that require fine-grained tuning.

Effective leaders provide more precise directional guidance; in optimization problems where identifying the correct path to the global optimum relies heavily on accurate navigation, enhancements to the leader mechanism significantly improve the probability of converging to high-quality regions. Furthermore, adjustments to the update rules reduce excessive fluctuations in position changes during the final iterations, thereby increasing stability and enabling the algorithm to achieve substantially more accurate convergence, as evidenced by reduced mean fitness values across benchmark tests.

It is important to note that the integrated Proposed Method does not always outperform PM-Update in every benchmark. Combining two strong components—one aimed at enhancing exploration and the other at improving exploitation—may induce interference under certain parameter settings. Consequently, the performance of the hybrid variant is sensitive to the relative weighting, operator sequencing, and control parameters that govern the two components. It is essential to carefully calibrate these factors to ensure optimal performance.

The ablation results indicate that leader-update refinement (PM-Update) is the main driver of the observed performance improvements. It leads to significant reductions in mean fitness and higher success rates across most tasks. The exploration-oriented module (PM-Exp) adds considerable value in problems with widely dispersed optima by maintaining or even enhancing population diversity. Generally, the proposed method (RL-LGWO) demonstrates a balanced and strong overall performance. However, its effectiveness relies on suitable parameterization and adaptive mechanisms to minimize potential functional interference between the components.

Table 7
Comparative results on benchmark functions F1–F23 for each proposed component, the baseline, and the complete proposed method.

Function	PM_Exp	PM_Update	GWO	Proposed method
F1	$1.42E-46$	**1.62E−186**	$2.59E-31$	$3.92E-105$
F2	$2.00E-29$	**4.66E−101**	$1.86E-18$	$8.20E-59$
F3	$8.60E-07$	**6.21E−39**	$1.29E-04$	$1.32E-32$
F4	$5.66E-04$	**1.46E−68**	$2.53E-06$	$5.49E-60$
F5	**2.61E+01**	$2.72E+01$	$2.69E+01$	**2.66E+01**
F6	$1.25E+00$	**2.57E−01**	$7.16E-01$	$9.82E-01$
F7	$7.94E-04$	**7.77E−04**	$2.14E-03$	$1.67E-03$
F8	**−6.47E+03**	$-5.67E+03$	$-5.95E+03$	$-5.76E+03$
F9	$5.68E-14$	**0.00E+00**	$1.03E+01$	**0.00E+00**
F10	$1.47E-14$	**4.44E−16**	$6.39E-14$	$1.33E-15$
F11	**0.00E+00**	**0.00E+00**	$5.58E-03$	**0.00E+00**
F12	$1.29E-01$	**1.36E−02**	$4.96E-02$	$5.72E-02$
F13	$5.50E-01$	**5.16E−01**	$6.61E-01$	$8.18E-01$
F14	$2.98E+00$	**9.98E−01**	$4.77E+00$	$4.62E+00$
F15	$5.08E-04$	**4.17E−04**	$8.37E-03$	**4.17E−04**
F16	**−1.03E+00**	**−1.03E+00**	**−1.03E+00**	**−1.03E+00**
F17	**3.98E−01**	**3.98E−01**	**3.98E−01**	**3.98E−01**
F18	**3.00E+00**	**3.00E+00**	**3.00E+00**	**3.00E+00**
F19	**−3.86E+00**	**−3.86E+00**	**−3.86E+00**	**−3.86E+00**
F20	$-3.20E+00$	**−3.32E+00**	$-3.26E+00$	$-3.26E+00$
F21	$-8.90E+00$	**−9.15E+00**	$-8.89E+00$	$-8.95E+00$
F22	$-9.00E+00$	**−1.04E+01**	$-9.49E+00$	$-1.01E+01$
F23	$-1.02E+01$	**−1.05E+01**	**−1.05E+01**	**−1.05E+01**

4.6. Convergence analysis

The convergence curves for algorithms are depicted in Fig. 6, where they are averaged. Selected benchmark functions are shown for comparison, rather than including all functions. The convergence analysis effectively demonstrates the balance between the exploration and exploitation phases of the proposed algorithm. By analyzing the convergence plots, it becomes evident that the algorithm successfully prevents entrapment in local optima using the feedback provided by the generated data. This mechanism enables agents to improve search efficiency and effectively identify optimal points in the search space.

For unimodal functions, which contain only one global optimum, the proposed method achieves faster convergence compared to other algorithms. In multi-modal functions, such as F9 and F11, the algorithm demonstrates its capability to escape local optima efficiently, eventually converging to the global optimum more effectively than competing methods. In F16, the algorithm does not converge immediately during the early iterations; instead, it explores the surrounding space and converges to the optimal solution in later iterations. This behavior highlights its ability to balance exploration and exploitation dynamically. Similarly, in F21 and F22, the algorithm avoids premature convergence and eventually outperforms other algorithms by achieving superior accuracy through continued exploration and search refinement.

As indicated by the convergence curves and accuracy tables, the proposed method consistently achieves better results compared to algorithms that prematurely converge. This demonstrates its robustness and ability to strike a balance between the exploration and exploitation phases, ensuring it avoids local optima while achieving the global optimum effectively.

In conclusion, the convergence analysis confirms the proposed algorithm's capability to adaptively balance exploration and exploitation, leading to superior performance across various benchmark functions.

4.7. Stability analysis

To further examine the stability of the algorithm's performance, we conducted an analysis of data dispersion using box plots, as depicted

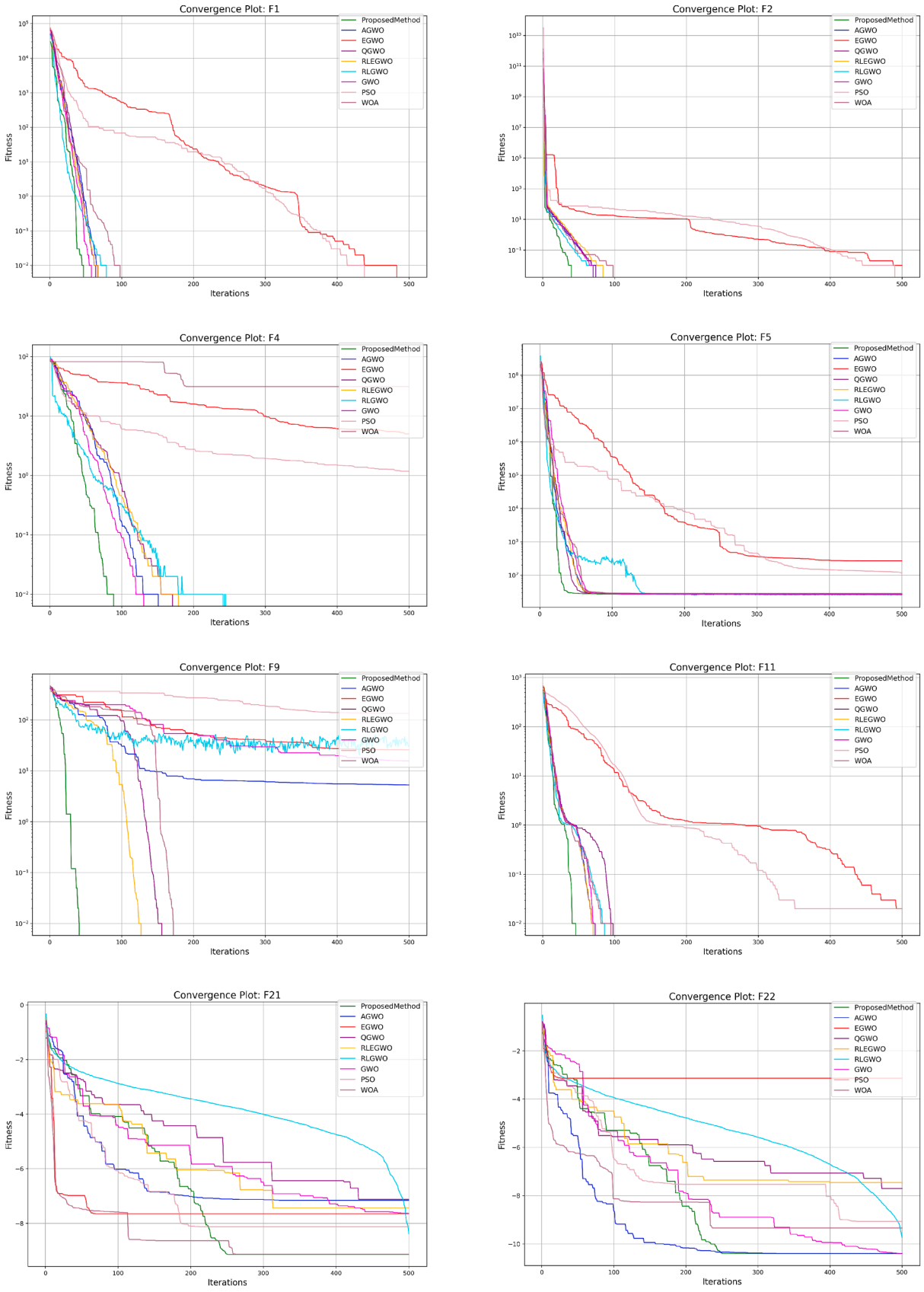


Fig. 6. The convergence curves in 500 iterations.

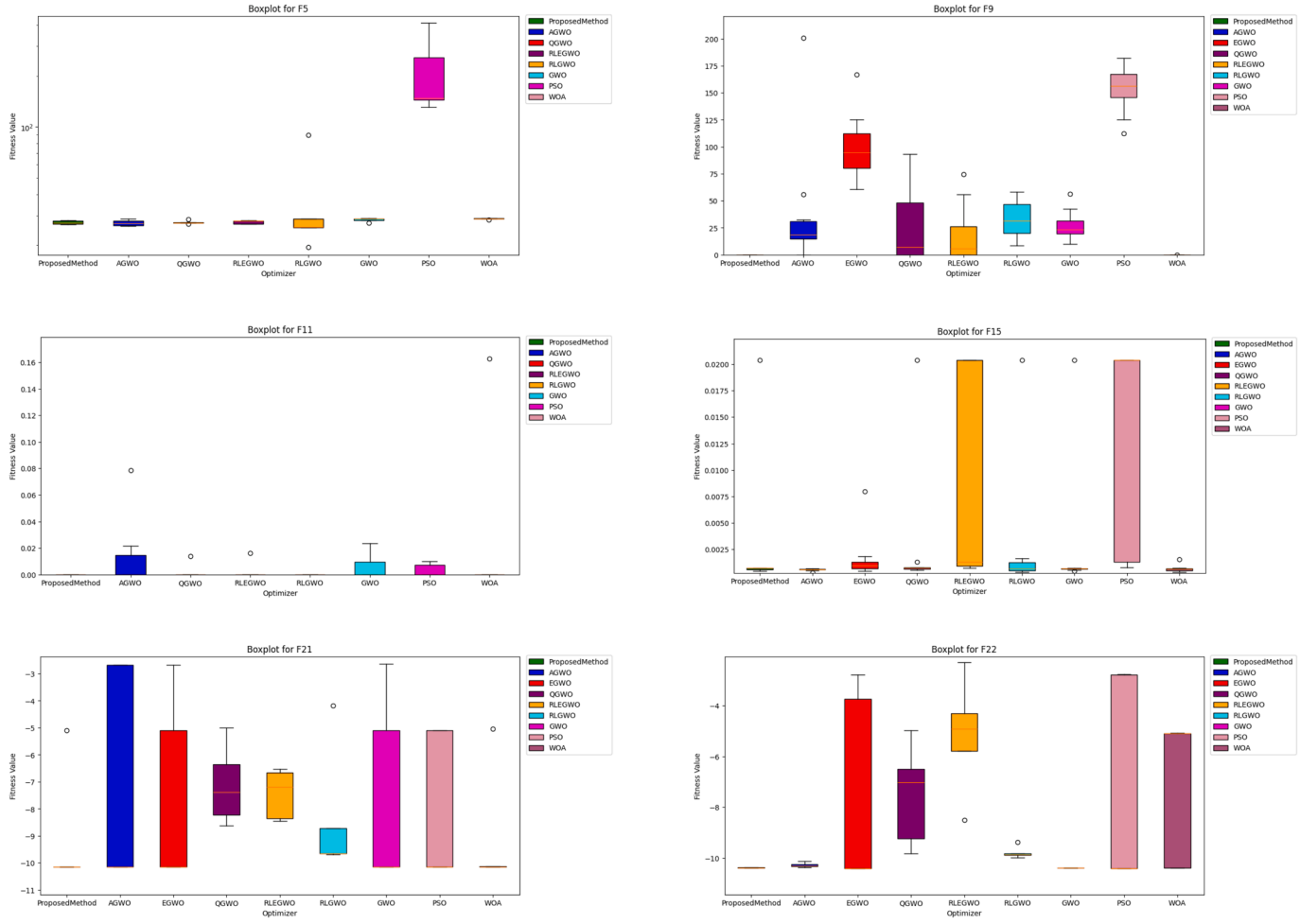


Fig. 7. The boxplots for 20 independent runs.

Table 8
Wilcoxon signed-rank test between the **Proposed** method and other algorithms over F1–F23. H=1 means significant at $\alpha = 0.05$. $S = \{+, -, =\}$ indicates whether Proposed is better, worse, or tied. Last row: overall wins/ties/losses (w/t/l).

No.	AGWO			EGWO			QGWO			RLEGWO			RLGWO			GWO			PSO			WOA		
	P	H	S	*P*	H	S	*P*	H	S	*P*	H	S	*P*	H	S	*P*	H	S	*P*	H	S	*P*	H	S
F1	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+
F2	8.84E−05	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	8.83E−05	1	+	1.91E−06	1	+	1.91E−06	1	+
F3	8.80E−05	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	2.67E−05	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+
F4	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+
F5	0.7841	0	=	1.91E−06	1	+	0.4090	0	=	0.6742	0	=	0.0012	1	-	0.6215	0	=	0.0001	1	+	0.0003	1	+
F6	0.0012	1	–	1.91E−06	1	–	0.5458	0	=	0.0027	1	+	0.8123	0	=	0.0014	1	–	1.91E−06	1	–	6.29E−05	1	–
F7	3.81E−06	1	+	1.91E−06	1	+	0.5958	0	=	0.8694	0	=	0.0094	1	+	0.0826	0	=	1.91E−06	1	+	0.1649	0	=
F8	0.0973	0	=	1.91E−06	1	–	3.81E−06	1	+	1.91E−06	1	+	0.6215	0	=	0.9854	0	=	3.62E−05	1	+	3.81E−06	1	–
F9	8.81E−05	1	+	1.91E−06	1	+	0.0002	1	+	0.0007	1	+	0.0003	1	+	1.91E−06	1	+	1.91E−06	1	+	0.0883	0	=
F10	8.84E−05	1	+	1.91E−06	1	+	7.74E−05	1	+	7.12E−05	1	+	1	0	=	8.71E−05	1	+	1.91E−06	1	+	0.0040	1	+
F11	1.91E−06	1	+	1.91E−06	1	+	0.0678	0	=	0.3173	0	=	0.0179	1	+	0.3173	0	=	1.91E−06	1	+	0.3173	0	=
F12	0.0003	1	–	0.3682	0	=	0.0695	0	=	0.0120	1	+	0.9854	0	=	0.0048	1	–	3.62E−05	1	–	1.91E−06	1	–
F13	0.0002	1	–	0.2161	0	=	0.4304	0	=	0.6742	0	=	2.67E−05	1	+	0.0023	1	–	1.91E−06	1	–	6.29E−05	1	–
F14	0.4321	0	=	0.3917	0	=	0.7603	0	=	0.7647	0	=	0.0436	1	+	0.8876	0	=	0.1009	0	=	0.0733	0	=
F15	0.0239	1	+	0.0014	1	+	0.0048	1	+	0.0010	1	+	0.0399	1	+	0.0063	1	+	0.0002	1	+	0.0083	1	+
F16	8.83E−05	0	=	8.83E−05	0	=	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	0.0973	0	=	8.83E−05	0	=	8.83E−05	0	=
F17	0.7475	0	=	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	3.81E−06	1	+	0.3341	0	=	1.91E−06	0	=	0.3117	0	=
F18	0.0010	1	+	1.91E−06	1	–	0.3682	0	=	0.1536	0	=	0.0072	1	–	0.1327	0	=	1.91E−06	1	–	0.9854	0	=
F19	0.2610	0	=	1.91E−06	1	–	0.1053	0	=	0.3883	0	=	0.8408	0	=	0.0327	0	=	0.0072	0	=	0.2610	0	=
F20	0.2610	0	=	0.7011	0	=	0.2161	0	=	0.2024	0	=	0.4090	0	=	0.0362	0	=	0.6476	0	=	0.0094	1	+
F21	0.2610	0	=	0.0266	1	+	0.0327	1	+	0.0171	1	+	0.8408	0	=	0.0362	1	+	0.5458	0	=	0.1893	0	=
F22	0.0007	1	+	0.0239	1	+	1.91E−06	1	+	1.91E−06	1	+	1.91E−06	1	+	0.0031	1	+	0.2610	0	=	3.62E−05	1	+
F23	0.0007	1	+	0.0008	1	+	0.0192	1	+	0.0019	1	+	0.0973	0	=	0.0055	1	+	0.3883	0	=	0.0031	1	+
w/t/l	14/7/2			15/5/3			13/8/2			12/9/2			11/10/2			16/4/3			14/6/3			13/7/3		

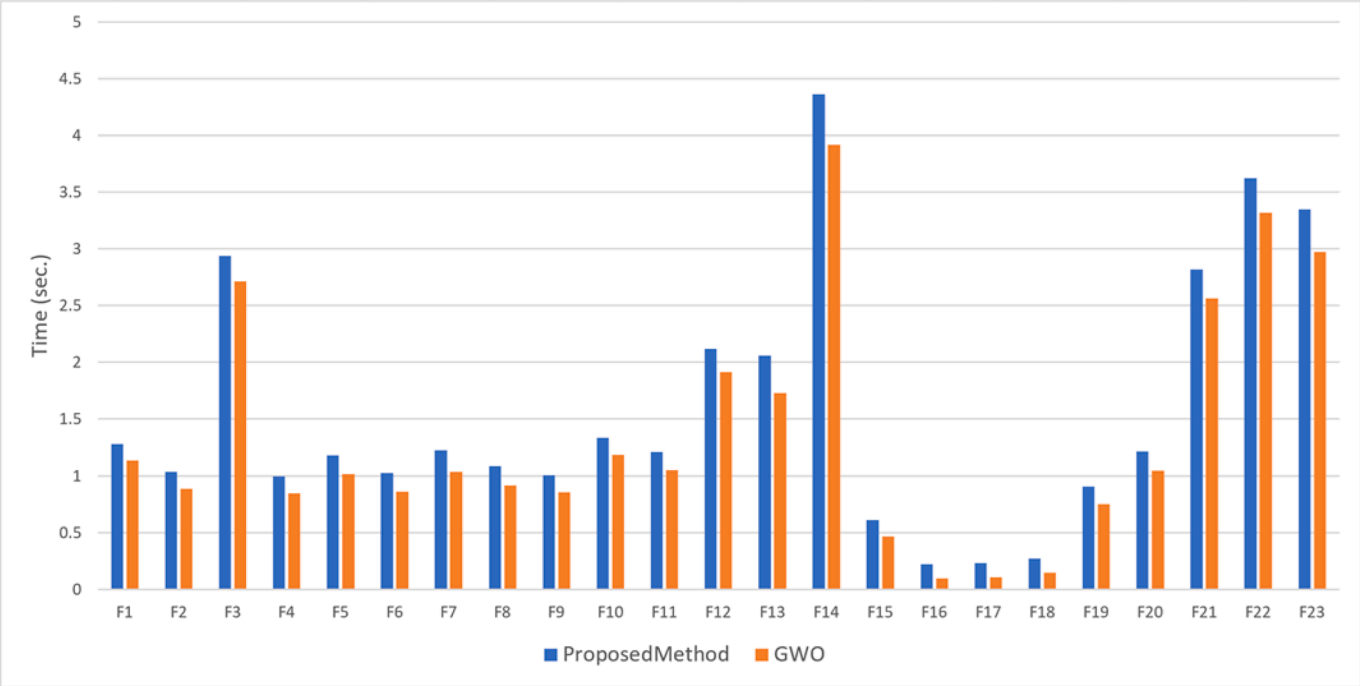


Fig. 8. Comparison of the running time between RL-LGWO and the original GWO algorithm.

in Fig. 7. Boxplots are employed to provide a detailed analysis of the dispersion, stability, and robustness of the results produced by various optimization algorithms across benchmark functions. This statistical visualization allows us to assess the consistency of each algorithm by examining key parameters such as the spread of data (range), the median value, and the presence of outliers.

As illustrated in the figure, the proposed algorithm consistently demonstrates a significantly lower standard deviation compared to the other algorithms across the benchmark functions. This highlights its stable and reliable performance. In contrast, the other algorithms only manage to achieve low standard deviations on certain benchmark functions, indicating their performance may be less consistent or dependent on the specific characteristics of those functions.

Furthermore, to verify whether the improvements of the proposed algorithm over the competitors are statistically significant, the Wilcoxon signed-rank test at a significance level of $\alpha = 0.05$ was conducted. The detailed results are summarized in Table 8. In the table, the p-value indicates the statistical evidence of difference, H represents the hypothesis test outcome (1 = significant, 0 = not significant), and S denotes the comparison result between the proposed algorithm and each competitor (‘ + ’: better, ‘−’: worse, ‘ = ’: similar).

As can be seen, for the majority of benchmark functions the p-value is less than 0.05, confirming that the improvements achieved by the proposed algorithm are statistically significant. In some cases (e.g., F19–F23), the p-value is larger than 0.05, which implies no statistically significant difference. This can be explained by the fact that in these functions all algorithms converge very closely to the optimal value, making the differences in performance too small to be considered significant. According to the win/tie/loss (w/t/l) summary, the proposed algorithm outperforms most of the competing algorithms on the majority of benchmark functions, which confirms its superior accuracy and robustness.

4.8. Running time

Fig. 8 presents a comparison of the running time between RL-LGWO and the original GWO algorithm. The results demonstrate that while RL-LGWO significantly enhances the accuracy, stability, and convergence rate, it introduces only a minimal computational overhead compared to the original GWO. This balance between performance improvement and computational efficiency highlights RL-LGWO’s suitability for tackling complex optimization problems, where both accuracy and efficiency are critical.

5. Conclusion and future work

This paper proposed a novel approach for enhancing leadership-based metaheuristic algorithms using reinforcement learning, with a focus on the GWO algorithm. By incorporating reinforcement learning into the leaders-followers mechanism of GWO, the proposed method effectively utilizes the data generated during the optimization process to achieve a balance between exploration and exploitation. Also, the adaptive updates of leaders through reinforcement learning help them avoid getting trapped in local optima and improve the overall optimization performance. In this study, the proposed algorithm outperformed other methods in terms of accuracy, achieving better results in 17 cases. Additionally, the stability of the results was significantly improved. These findings highlight the algorithm’s ability to provide more accurate and consistent solutions across varying conditions. Experimental results show that the proposed algorithm, RL-LGWO, has a minor runtime increase compared to the original GWO but significantly outperforms other methods.

In conclusion, the integration of reinforcement learning as a built-in component proved highly effective in guiding the algorithm using the generated data, leading to significant improvements in optimization performance. Moreover, this framework is generalizable and can be applied to other leader-follower metaheuristic algorithms, offering a promising direction for future research in this field. The proposed method was designed to enhance leader-follower-based metaheuristic algorithms.

Future research can extend this approach to other algorithms, such as PSO, ACO, or WOA.Reporting and analyzing the results of these experiments can reveal additional strengths or areas for refinement.

While the algorithm demonstrated excellent performance on benchmark functions, future studies could investigate its application to complex real-world optimization problems. Examples include logistics and supply chain management, engineering design optimization, financial modeling, and healthcare scheduling.

Currently, the proposed method leverages reinforcement learning to share information among agents and improve exploration. Future work could explore alternative mechanisms for agent communication and information sharing.

Also, investigating the scalability of the proposed method on high-dimensional problems can provide insights into its limitations and potential improvements. This includes analyzing its computational complexity and exploring optimization techniques to make it more efficient for large-scale problems.

Data availability

The authors declare that the data supporting the findings of this study are available within the paper.

CRediT authorship contribution statement

Afifeh Maleki: Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Mehdy Roayaei:** Writing – review & editing, Validation, Methodology, Formal analysis, Conceptualization; **Seyedali Mirjalili:** Writing – review & editing, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

References

[1] C. Blum, A. Roli, Metaheuristics in combinatorial optimization, ACM Comput. Surv. (CSUR) 35 (3) (2003) 268–308. https://doi.org/10.1145/937503.937505
[2] H. Faris, I. Aljarah, M.A. Al-Betar, S. Mirjalili, Grey wolf optimizer: a review of recent variants and applications, Neural Comput. Appl. 30 (2) (2018) 413–435. https://doi.org/10.1007/S00521-017-3272-5/TABLES/4
[3] Y. Qiu, X. Yang, S. Chen, An improved gray wolf optimization algorithm solving to functional optimization and engineering design problems, Sci. Rep. 14 (1) (2024) 1–24. https://doi.org/10.1038/s41598-024-64526-2
[4] Z. Al-Ibraheemi, S. Al-Janabi, Sustainable energy: advancing wind power forecasting with grey wolf optimization and GRU models, Results Eng. 24 (2024) 102930. https://doi.org/10.1016/j.rineng.2024.102930
[5] Y. Liu, Y. Zhang, A leader-based particle swarm optimization algorithm for solving combinatorial optimization problems, Swarm Evol. Comput. 27 (2016) 71–83.
[6] G. Dhiman, V. Kumar, Spotted hyena optimizer: a novel bio-inspired based metaheuristic technique for engineering applications, Adv. Eng. Softw. 114 (2017) 48–70. https://doi.org/10.1016/J.ADVENGSOFT.2017.05.014
[7] H. Zhang, J. Sun, T. Liu, K. Zhang, Q. Zhang, Balancing exploration and exploitation in multiobjective evolutionary optimization, Inf. Sci. 497 (2019) 129–148. https://doi.org/10.1016/j.ins.2019.05.046
[8] M. Črepinšek, S.-H. Liu, M. Mernik, Exploration and exploitation in evolutionary algorithms: a survey, ACM Comput. Surv. (CSUR) 45 (3) (2013) 1–33. https://doi.org/10.1145/2460276.2460277
[9] J. Hu, A.A. Heidari, L. Zhang, X. Xue, W. Gui, H. Chen, Z. Pan, Chaotic diffusion-limited aggregation enhanced grey wolf optimizer: insights, analysis, binarization, and feature selection, Int. J. Intell. Syst. 37 (8) (2022) 4864–4927. https://doi.org/10.1002/int.22744
[10] S. Gupta, K. Deep, Enhanced leadership-inspired grey wolf optimizer for global optimization problems, Eng. Comput. 36 (4) (2019) 1777–1800. https://doi.org/10.1007/S00366-019-00795-0
[11] J. Chen, H. Heidari, H. Chen, Data-driven adaptive evolutionary algorithms: a survey, IEEE Trans. Evol. Comput. 23 (6) (2019) 1025–1038. https://doi.org/10.1109/TEVC.2019.2915312
[12] N. Mazyavkina, S. Sviridov, S. Ivanov, E. Burnaev, Reinforcement learning for combinatorial optimization: a survey, Comput. Oper. Res. 134 (2021) 2021. https://doi.org/10.1016/j.cor.2021.105400
[13] M. Ghalambaz, R.J. Yengejeh, A.H. Davami, Building energy optimization using grey wolf optimizer (GWO), Case Stud. Therm. Eng. 25 (2021) 101250. https://doi.org/10.1016/j.csite.2021.101250
[14] F. Kiani, A. Seyyedabbasi, R. Aliyev, M.U. Gulle, M.A. Shah, 3D Path planning method for multi-UAVs inspired by grey wolf algorithms, J. Internet Technol. 22 (4) (2021) 743–755. https://doi.org/10.53106/160792642021072204003
[15] F. Kiani, A. Seyyedabbasi, P. Mahouti, Optimal characterization of a microwave transistor using grey wolf algorithms, Analog Integr. Circuits Signal Process. 109 (2021) 599–609. https://doi.org/10.1007/s10470-021-01863-4
[16] N. Yuvaraj, T. Karthikeyan, K. Praghash, An improved task allocation scheme in serverless computing using gray wolf optimization (GWO) based reinforcement learning (RIL) approach, Wirel. Pers. Commun. 117 (2020) 2403–2421. https://doi.org/10.1007/S11277-020-07981-0
[17] C.R. Lin, Z.C. Cao, M.C. Zhou, Learning-based grey wolf optimizer for stochastic flexible job shop scheduling, IEEE Trans. Autom. Sci. Eng. 19 (4) (2022) 3659–3671. https://doi.org/10.1109/TASE.2021.3129439
[18] E. Emary, H.M. Zawbaa, C. Grosan, Experienced gray wolf optimization through reinforcement learning and neural networks, IEEE Trans. Neural Netw. Learn. Syst. 29 (3) (2018) 681–694. https://doi.org/10.1109/TNNLS.2016.2634548
[19] A. Seyyedabbasi, R. Aliyev, F. Kiani, M.U. Gulle, H. Basyildiz, M.A. Shah, Hybrid algorithms based on combining reinforcement learning and metaheuristic methods to solve global optimization problems, Knowledge-Based Syst. 223 (2021). https://doi.org/10.1016/j.knosys.2021.107044
[20] M. Becerra-Rozas, et al., Reinforcement learning based whale optimizer, LNCS 12957 (2021) 205–219. https://doi.org/10.1007/978-3-030-87013-3_16
[21] F. Zhao, X. Hu, L. Wang, J. Zhao, J. Tang, Jonrinaldi, A reinforcement learning brain storm optimization algorithm (BSO) with learning mechanism, Knowledge-Based Syst. 235 (2022). https://doi.org/10.1016/J.KNOSYS.2021.107645
[22] A.K. Sadhu, A. Konar, T. Bhattacharjee, S. Das, Synergism of firefly algorithm and Q-learning for robot arm path planning, Swarm Evol. Comput. 43 (2018) 50–68. https://doi.org/10.1016/J.SWEVO.2018.03.014
[23] D. Wu, G.G. Wang, Employing reinforcement learning to enhance particle swarm optimization methods, Eng. Optim. 54 (2) (2022) 329–348. https://doi.org/10.1080/0305215X.2020.1867120
[24] D. Catteeuw, M. Drugan, B. Manderick, Guided' Restarts Hill–Climbing: a reinforcement learning approach, J. Heuristics 27 (1) (2021) 65–89. https://doi.org/10.1007/s10732-020-09459-1
[25] M. Becerra-Rozas, J.A. Soria-Alcaraz, C.A.C. Coello, Reinforcement learning based whale optimizer, LNCS 12957 (2021) 205–219. https://doi.org/10.1007/978-3-030-87013-3_16
[26] R. Zhang, H. Yu, K. Gao, Y. Fu, J.H. Kim, A Q-learning based artificial bee colony algorithm for solving surgery scheduling problems with setup time, Swarm Evol. Comput. 90 (2024) 101686. https://doi.org/10.1016/j.swevo.2024.101686
[27] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey wolf optimizer, Adv. Eng. Softw. 69 (2014) 46–61. https://doi.org/10.1016/J.ADVENGSOFT.2013.12.007
[28] J. Kennedy, R. Eberhart, Particle Swarm Optimization, 4 of ICNN'95-International Conference on Neural Networks, IEEE, 1995.
[29] S. Mirjalili, A. Lewis, The whale optimization algorithm, Adv. Eng. Softw. 95 (2016) 51–67.
[30] K. Meidani, A.P. Hemmasian, S. Mirjalili, A. Barati, Farimani, Adaptive grey wolf optimizer, Neural Comput. Appl. 34 (10) (2022) 7711–7731. https://doi.org/10.1007/S00521-021-06885-9
[31] K. Luo, Enhanced grey wolf optimizer with a model for dynamically estimating the location of the prey, Appl. Soft Comput. 77 (2019) 225–235. https://doi.org/10.1016/J.ASOC.2019.01.025
[32] C. Qu, W. Gai, M. Zhong, J. Zhang, A novel reinforcement learning based grey wolf optimizer algorithm for unmanned aerial vehicles (UAVs) path planning, Appl. Soft Comput. 89 (2020) 106099. https://doi.org/10.1016/J.ASOC.2020.106099
[33] M. Zhao, R. Hou, H. Li, M. Ren, A hybrid grey wolf optimizer using opposition-based learning, sine cosine algorithm and reinforcement learning for reliable scheduling and resource allocation, J. Syst. Softw. 205 (2023) 111801. https://doi.org/10.1016/J.JSS.2023.111801
[34] Y. Zhang, Z. Jin, Y. Chen, Hybridizing grey wolf optimization with neural network algorithm for global numerical optimization problems, Neural Comput. Appl. 32 (2019) 10451–10470. https://doi.org/10.1007/S00521-019-04580-4
[35] M. Roayaei, RLSetGWO. GitHub Repository, 2025. https://github.com/mroayaei/RLSetGWO.git.
[36] A. Kumar, K.V. Price, A.W. Mohamed, A.A. Hadi, P.N. Suganthan, Problem Definitions and Evaluation Criteria for the 2022 Special Session and Competition on Single Objective Bound Constrained Numerical Optimization, Technical Report, Nanyang Technological University, Singapore, 2021.
[37] F. Kiani, A. Seyyedabbasi, S. Nematzadeh, Improving the performance of hierarchical wireless sensor networks using the metaheuristic algorithms: efficient cluster head selection, Sens. Rev. 41 (4) (2021) 368–381.
[38] S. Rajakumar, P.S.S. Sreedhar, S. Kamatchi, G. Tamilmani, Gray wolf optimization and image enhancement with NLM algorithm for multimodal medical fusion imaging system, Biomed. Signal Process. Control 85 (2023), 104950.
[39] J. Hu, H. Chen, A.A. Heidari, M. Wang, X. Zhang, Y. Chen, Z. Pan, Orthogonal learning covariance matrix for defects of grey wolf optimizer: insights, balance, diversity, and feature selection, Knowledge-Based Syst. 213 (2021), 106684.